Beginner Homelab on a Raspberry Pi

Self-host your media, block ads network-wide, get a one-URL control panel, reach everything from your phone, and route your devices through a VPN, all over a private Tailscale network

Evanns Morales-Cuadrado

2026-05-31

Table of contents

Preface. What this series builds

This is a beginner-friendly guide to standing up a small **homelab** on a Raspberry Pi, organized into two **volumes** of short chapters. You start by building a foundation, then give it one job at a time. By the end you have one always-on Pi that serves your media, cleans the ads off every device you own, hands you a single dashboard URL, lets you reach everything from your phone, and can route your traffic through a VPN. And *every* piece is reachable from anywhere over a private Tailscale network with nothing exposed to the public internet.

Volume I: Building the homelab

1. **Foundation: your Pi and your Tailscale network.** Flash a headless 64-bit Linux with key-based SSH, install Docker, and put the Pi, your laptop, and your phone on one private Tailscale network with MagicDNS, reachable by name from anywhere.
2. **Streaming audiobooks with Audiobookshelf.** Serve spoken audio from the Pi, streaming to your phone with proper resume tracking. The running example is my own: a language course I ripped from CD so I can study from anywhere without carrying the discs.
3. **Network-wide ad blocking with Pi-hole.** Run Pi-hole as a DNS server and point your router at it, so every device on your home network is ad-blocked with zero per-device setup.
4. **Pretty URLs: a reverse proxy and local DNS.** Add Caddy and local DNS so you can type `https://pihole.home` and `https://abs.home` on your tailnet, no ports or IPs, with real HTTPS.
5. **A one-URL dashboard.** Add Homepage and Portainer, slotted into the proxy at `https://home.home` as a single landing page.
6. **Remoting into your machines from your phone.** Reach your Pi's terminal (Termius) and a full desktop (NoMachine) from your phone, by Tailscale machine name.

Volume II: On the road and your devices

7. **VPN privacy: Aura, Mullvad, and your options.** What a VPN does, how Aura compares to Mullvad, and what a Tailscale exit node is, so you can pick the approach that fits.
8. **On the road: homelab, Pi-hole, and VPN away from home.** The field manual for leaving the house, and the one rule for why homelab access, Pi-hole, and a privacy VPN can't all run at once.
9. **Wiring your phone into your Linux machines.** LocalSend and KDE Connect for file transfer and clipboard sharing over Tailscale, plus the USB fallback for bulk photo offload.

Two **appendices** back the parts up: Appendix A explains every Docker Compose file line by line (kept out of the main chapters so they stay readable), and Appendix B is a "what should just work" reference, every URL and the one-line check that proves it's healthy, ideal as a post-reboot smoke test.

The hardware and accounts

- **The always-on server.** I'm running this on a **Raspberry Pi 4 Model B (8 GB RAM)** with a **32 GB microSD card**, in an **Argon ONE M.2 Aluminum case** (passive-plus-fan cooling, with an M.2 slot for a future SSD), on **Ubuntu Server 26.04 LTS (64-bit), headless** (no desktop, just SSH). That's simply what I happen to use, and what these steps are written and tested on. You don't need the exact same kit; anything similar (a Pi 4 or 5 with a few GB of RAM and a 16 GB+ card, on a Debian/Ubuntu base) will follow along. There's plenty of headroom here, since all the containers, configs, and a ripped audio course together use only a few GB.
- **A laptop/desktop** (Linux assumed) to flash the card and use as a client.
- **A free Tailscale account.** One account, signed into on every device, is the connective tissue for the entire series.
- **An iPhone** (the apps in Chapters 2 and 9 also exist for Android; the commands are identical).
- **Docker** on the Pi, installed once in Chapter 1, reused by every later chapter.
- A Mullvad subscription *or* the Tailscale Mullvad add-on for Chapter 7 (both optional, ~$5/month; Chapter 7 helps you choose).

Conventions used throughout

- I'm running **Ubuntu Server 26.04 LTS** on the Pi, which is a **Debian/Ubuntu-flavored** Linux, so commands use `apt`, `systemd`, and `docker`.
- Commands prefixed with `sudo` need administrator rights; the rest run as your normal user.
- **Placeholders, substitute your own.** Wherever you see a Tailscale address like `100.x.y.z`, run `tailscale ip -4` on that machine to get the real one. This guide names the Raspberry Pi `homelab` on the tailnet; its full DNS name is therefore `homelab.<your-tailnet>.ts.net`, where `<your-tailnet>` is the suffix shown on the **DNS** page of the Tailscale admin console (something like `tail1a2b3.ts.net`). Substitute it wherever you see `your-tailnet.ts.net`. Likewise, `/home/you/` stands in for your own home directory, and `you@homelab` for your own username on the Pi.

> **i** Why Tailscale is the spine of this series
>
> Every part avoids port forwarding, dynamic DNS, and exposing services to the public internet. Instead, each service binds to the Pi and is reached over an authenticated WireGuard mesh. The practical upshot: your homelab is private by default, works on locked-down corporate Wi-Fi, and needs no router configuration. You sign in once per device and everything just finds everything else.

! Security & secrets: read before you publish or push anything

This guide is written to be **safe to commit to a public Git repo** and safe to follow. Two habits make that true, and they're applied throughout:

- **Never put a real password, API token, or key in a file you commit.** Every config below reads its secrets from a local `.env` file that is listed in `.gitignore` and never leaves your machine.
- **Nothing is exposed to the public internet.** Services are reached only over your private tailnet or your home LAN, never by port-forwarding. Remote access to the Pi uses SSH over Tailscale with **key-based authentication** (set up in Chapter 1). Don't open router ports to any of this.

If you fork this guide for your own site, scrub any real tailnet names, IPs, and usernames first, they're mildly identifying. This guide uses placeholders for exactly that reason.

Part I

Volume I: Building the homelab

1 Chapter 1. Foundation: your Raspberry Pi and your Tailscale network

> **The payoff of this chapter:** an always-on Raspberry Pi that's ready to host everything in this series, running Docker, hardened SSH, and joined to a private Tailscale network, plus your laptop and iPhone on that same network, so every machine can reach every other by name, from anywhere, with nothing exposed to the public internet.

This is the groundwork the rest of the book stands on. We don't install any "app" here, instead we build the platform: a Pi that's online, secured, and addressable, and a **tailnet** (your private Tailscale network) that stitches your devices together. Every later chapter (Audiobookshelf, Pi-hole, the dashboard, the VPN) simply plugs into what you set up now.

1.1 Why Tailscale is the spine of everything

Tailscale is a zero-config mesh VPN built on WireGuard. Once installed on your devices and signed into one account, each device gets a stable private address (a `100.x.y.z`) and can reach the others directly, on any network, with no port forwarding and nothing open to the internet.

Why this instead of port forwarding:

- Port forwarding needs a public IP, router configuration, dynamic DNS if your ISP rotates addresses, and it exposes your services to the entire internet.
- A relay like Cloudflare Tunnel avoids port forwarding but proxies all your traffic through a third party.
- Tailscale punches out from both ends and builds a direct, encrypted WireGuard tunnel. Your Pi never accepts an inbound connection from the public internet, only devices on *your* tailnet can talk to it.

That last point is why this guide never tells you to open a router port: the homelab is private by default and works even on locked-down corporate Wi-Fi.

1.2 Prerequisites

- **The always-on server.** I'm using a **Raspberry Pi 4 Model B (8 GB)** with a **32 GB microSD card**, in an **Argon ONE M.2 Aluminum case**, on **Ubuntu Server 26.04 LTS (64-bit), headless**, plus a power supply. That's what these steps are written and tested on. You can follow along on anything similar (a Pi 4 or 5 with a few GB of RAM and a 16 GB+ card).

- A **laptop/desktop** (Linux assumed; the commands use `apt`/`systemd`) to flash the card and to use as a client.
- An **iPhone** (the apps used later also exist for Android; commands are identical).
- A **free Tailscale account**, sign up with Google/GitHub/Microsoft or email at https://tailscale.com. One account, signed into on every device, is the only “server” you need.

1.3 Part A: Flash and first boot

1.3.1 Step 1: Flash a headless 64-bit Linux with key-based SSH

Install **Raspberry Pi Imager** on your laptop. I’m running **Ubuntu Server 26.04 LTS (64-bit)**, so in Imager I choose **Other general-purpose OS → Ubuntu → Ubuntu Server 26.04 LTS (64-bit)**. It’s headless, just SSH and a shell, which is all a server needs.

Before writing the card, open Imager’s settings (the gear / **Edit settings**):

- Set a **hostname** (e.g. `homelab`) and your **username**.
- Under **Services**, enable **SSH → “Allow public-key authentication only”**, and paste your laptop’s public key. If you don’t have one yet, generate it first:

  ```
  ssh-keygen -t ed25519        # then paste the contents of
  ~/.ssh/id_ed25519.pub
  ```

This bakes key-based login in from first boot and disables password SSH, the single most important hardening step for a box that’s always on.

> **!** Remoting safety for an always-on server
>
> You’ll SSH into this Pi for the rest of the series, so set it up safely once:
>
> - **Key-based auth only, no password login.** The Imager option above handles it; on an already-running Pi you’d `ssh-copy-id you@homelab`, then set `PasswordAuthentication no` in `/etc/ssh/sshd_config` and `sudo systemctl restart ssh`.
> - **Reach it over Tailscale, never the public internet.** Once the Pi is on your tailnet (Step 3) you SSH to it as `you@homelab` from anywhere, there is never a reason to port-forward SSH (or anything) on your router. An open port 22 draws constant brute-force traffic; Tailscale sidesteps it entirely.
> - **Keep it patched:** `sudo apt update && sudo apt upgrade -y` periodically, or enable `unattended-upgrades`.

Write the card, put it in the Pi, and power on.

1.3.2 Step 2: First boot, then install Docker

Find the Pi on your LAN and SSH in (`ssh you@<pi-lan-ip>`, or `ssh you@homelab.local` if your network supports mDNS). Update it and install Docker, which every later chapter uses to run its services:

```
# Update the base image
sudo apt update && sudo apt upgrade -y

# Docker: the official one-line installer
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker        # apply the group change in this shell
```

> **Why Docker**
>
> Every service in this series (Audiobookshelf, Pi-hole, the dashboard, the reverse proxy) ships as a Docker container. Running them in containers keeps their dependencies off your host, makes upgrades a single `docker pull`, and lets each one live in its own tidy folder with a `compose.yaml`. Installing it once here means every later chapter just runs `docker compose up`.

1.4 Part B: Join the tailnet

1.4.1 Step 3: Put the Pi on your tailnet

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

`tailscale up` prints a URL, open it, sign in, and the Pi joins your tailnet. Check its address:

```
tailscale ip -4      # something like 100.x.y.z
```

That `100.x.y.z` is how every other device will reach the Pi. You won't have to memorize it, though, MagicDNS (next step) gives it a name.

1.4.2 Step 4: Name the Pi `homelab` and turn on MagicDNS

MagicDNS gives every device on your tailnet a stable name, so you can type `homelab` instead of an IP. In the Tailscale admin console (https://login.tailscale.com):

1. **DNS** page → confirm **MagicDNS** is enabled (it's on by default for tailnets created after late 2022; the button reads "Disable MagicDNS" when on). Note your **tailnet name** here, something like `tail1a2b3.ts.net`; wherever this guide writes `your-tailnet.ts.net`, substitute it.
2. **Machines** page → the menu on the Pi's row → **Edit machine name** → set it to `homelab`.

Now the Pi answers to `homelab` (and its full name `homelab.your-tailnet.ts.net`) from any device on the tailnet. Because MagicDNS installs your tailnet name as a **search domain**, the short name `homelab` usually works on its own; the full `homelab.your-tailnet.ts.net` is the always-reliable fallback.

1.5 Part C: Add your devices and verify

1.5.1 Step 5: Install Tailscale on your laptop

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up        # sign in with the SAME account
```

Your tailnet is one shared namespace, signing in with the same account simply adds the laptop as another peer. From now on you can `ssh you@homelab` from the laptop over the tailnet, anywhere.

1.5.2 Step 6: Install Tailscale on your iPhone

Install **Tailscale** from the App Store and sign in with the same account. You should see `homelab` (and your laptop) listed as peers. That's it, the phone is now on the mesh and can reach the Pi by name, on Wi-Fi or cellular.

1.5.3 Step 7: Verify the mesh

From the Pi (or your laptop):

```
tailscale status        # lists every device with a green marker when online
ping homelab            # from the laptop/phone, resolves via MagicDNS
ssh homelab             # from your laptop, log straight in by name
```

You should now be able to `ssh homelab` **directly**, no IP and no LAN address, from any device on your tailnet, anywhere. (That works as long as your laptop username matches the one on the Pi; if it differs, use `ssh you@homelab`. The first connection may ask you to confirm the host fingerprint, which is normal.)

If `homelab` resolves, `tailscale status` shows your laptop and phone online, and `ssh homelab` logs you in, the foundation is done.

A naming note you'll be glad to know later

`nslookup homelab` and `host homelab` can *fail* on macOS even when everything works, those tools bypass the resolver MagicDNS hooks into. Test with `ping homelab` or a browser instead, and keep `homelab.your-tailnet.ts.net` as the guaranteed-resolvable fallback. This trips people up; it's not a broken setup.

1.6 Recap

- **Flashed** a headless 64-bit Linux (Ubuntu Server 26.04 LTS) with key-only SSH (no passwords, no open ports).
- **Installed Docker** on the Pi, the runtime for every later service.
- **Built your tailnet:** the Pi, your laptop, and your iPhone all on one private WireGuard mesh, signed into one account.
- **Named the Pi `homelab`** via MagicDNS, so everything is reachable by name.

The platform is ready. In Chapter 2 we put it to work: ripping your Assimil discs and serving them from the Pi to your iPhone over the tailnet you just built.

2 Chapter 2. Streaming audiobooks to your iPhone with Audiobookshelf

The payoff of this chapter: a collection of spoken audio (audiobooks, podcasts, language courses, anything) copied once and served from the Raspberry Pi you set up in Chapter 1, streaming to your iPhone from anywhere over your tailnet, with proper audiobook-style resume tracking (pause mid-sentence today, resume at the same second tomorrow).

> ℹ The concept is general; the example is mine
>
> This part shows how to serve **any** spoken audio from your Pi. The running example I use throughout is my own use case: a language-learning audio course I ripped from CD, because what I actually wanted was to study on my lunch break with just the book and none of the discs. Wherever you see the course, picture your own audiobooks; the steps are the same.

In Chapter 1 you built the platform: a Pi named `homelab` running Docker and on your tailnet, reachable by name from your laptop and iPhone. Now we put it to work with **Audiobookshelf**, a self-hosted media server purpose-built for spoken audio. It runs in a Docker container, scans a folder of MP3s, and serves a web UI plus a JSON API consumed by official iOS and Android apps.

Why an audiobook server and not a music server: music servers (Navidrome, Plex, Jellyfin) track "what song you played last." Audiobook servers track "second 247 of track 14 of book X" and sync that position across every device. For anything where you pause mid-sentence and resume tomorrow, like a language course or a long audiobook, that's the whole point. So Audiobookshelf here is strictly for **spoken** audio: audiobooks, courses, and (later) podcasts. If I add music down the line it gets its own dedicated server, like **Navidrome**, in its own folder, not this one.

In my example the audio starts on CDs, so I need a CD/DVD drive somewhere. Use whatever you've got: your laptop or desktop's built-in drive, or a USB external plugged into the laptop or the Pi. It doesn't matter which, because the goal is the same either way, the audio ending up in **`~/audiobookshelf/media/Audiobooks` on the Pi**. If your audio is already files on disk, skip the ripping and just copy it into `~/audiobookshelf/media/Audiobooks` on the Pi (over the tailnet with `scp` or `rsync`, or off a USB stick).

2.1 Prerequisites

- You finished Chapter 1: the Pi (`homelab`) is on your tailnet with Docker installed, and your laptop and iPhone are on the tailnet too.

- A **CD/DVD drive with the disc inserted**, on whichever machine is handy: a built-in drive, or a USB external plugged into the laptop or the Pi. You'll do the ripping on that machine, then make sure the files land on the Pi.
- The Pi (`homelab`) reachable over SSH for the server steps.

  On the machine that has the drive, check it's seen:

  ```
  lsblk -o NAME,LABEL,FSTYPE,MOUNTPOINT | grep -i -E "rom|cdrom|sr"
  ```

  You should see `sr0`. If it isn't mounted (a headless Pi won't auto-mount it), mount it yourself; this prints the mount point (something like `/media/you/<LABEL>`):

  ```
  udisksctl mount -b /dev/sr0
  ```

2.2 Part A: Rip and copy the audio

2.2.1 Step 1: Inspect what's actually on the disc

Don't assume the layout. List the mount point (substitute the real label that `udisksctl` just printed):

```
ls /media/$USER/<disc-label>/
```

A language-course data CD like mine typically has three useful things:

1. **A folder of full lessons** with 100 files named `L001-LESSON.mp3` … `L100-LESSON.mp3`: one complete lesson per file, pre-assembled in playback order.
2. **Per-lesson folders** (`L001-…` … `L100-…`) that split each lesson into tiny per-sentence files. You can ignore these; the full lessons above are all you need.
3. **A User's Manual** in HTML and PDF.

Plus an `autorun.inf` (a Windows launcher you can ignore on Linux). It's a **data CD**, so this is the fast path: a plain file copy, no audio-CD ripping or re-encoding.

2.2.2 Step 2: Use the one-file-per-lesson folder

Copy the folder with **one complete lesson per file**. That way Audiobookshelf treats the course as a single audiobook with 100 sequential chapters: tap "next chapter" to advance one lesson, and your resume position lives at the lesson level, which is the right unit of progress for a language course.

2.2.3 Step 3: Get the audio onto the Pi

Because it's a data CD, "ripping" is just copying files off the mounted disc. The audiobook library lives at `~/audiobookshelf/media/Audiobooks` on the Pi (we keep the content tidy inside the Audiobookshelf folder, set up in Step 4). Do the copy on whichever machine has the drive.

2.2.3.1 3a. Copy the lessons

If the drive is on the Pi, copy straight into the library:

```
mkdir -p ~/audiobookshelf/media/Audiobooks/"My Course"
rsync -av --no-perms --progress \
  "/media/$USER/<disc-label>/<lessons-folder>/" \
  ~/audiobookshelf/media/Audiobooks/"My Course"/
```

`rsync` over `cp` because if the disc has a read error or you Ctrl-C halfway, re-running resumes where it stopped. `--no-perms` matters: a CD is mounted read-only (files at `r--------`), and the `-a` archive flag would copy those modes to the destination, leaving files you can barely read and directories you can't write into. `--no-perms` uses your umask defaults instead, so the result behaves like normal files you created. The `L001 … L100` names are already zero-padded, so Audiobookshelf orders them correctly.

2.2.3.2 3b. If the drive is on your laptop

Rip into any folder on the laptop, then push it to the library path on the Pi over the tailnet (works from anywhere, fastest on the same LAN):

```
rsync -av --no-perms --progress \
  "/media/$USER/<disc-label>/<lessons-folder>/" ~/"My Course"/
rsync -avz --partial --no-perms --progress \
  ~/"My Course"/ you@homelab:~/audiobookshelf/media/Audiobooks/"My Course"/
```

`-z` compresses over the wire and `--partial` keeps partial files so an interrupted transfer resumes quickly, useful for a GB-scale copy.

2.2.3.3 3c. Optional: save the manual

To keep the disc's User's Manual alongside the lessons, copy its folder into `~/audiobookshelf/media/Audiobooks/"` `Course/_manual"/` the same way. The leading underscore sorts it to the bottom so Audiobookshelf scans the lessons first.

2.2.3.4 3d. Unmount and eject

When the copy is done, unmount and eject on the machine that had the drive:

```
udisksctl unmount -b /dev/sr0
eject /dev/sr0
```

You only do this once; from now on the files live on the Pi's disk.

2.3 Part B: Run the server

2.3.1 Step 4: Run Audiobookshelf on the Pi

SSH into the Pi if you aren't already there (`ssh you@homelab`); Docker is installed from Chapter 1. We'll run Audiobookshelf as a small **Docker Compose** project, one folder with one `compose.yaml`, the same self-contained pattern every other service in this series uses (so it's one lifecycle to learn and a single file to back up).

```
mkdir -p ~/audiobookshelf/config ~/audiobookshelf/metadata    #
media/Audiobooks already exists from Step 3
cd ~/audiobookshelf

cat > compose.yaml <<'EOF'
services:
  audiobookshelf:
    container_name: audiobookshelf
    image: ghcr.io/advplyr/audiobookshelf:latest

    ports:
      - "13378:80"          # host 13378 -> container 80 (host 80 stays free
for Caddy)

    volumes:
      - ./media/Audiobooks:/audiobooks    # add ./media/Podcasts:/podcasts
here later
      - ./config:/config
      - ./metadata:/metadata

    restart: unless-stopped
EOF
```

Then keep the runtime data out of version control and launch:

```
printf 'config/\nmetadata/\nmedia/\n' > .gitignore
docker compose up -d
```

ℹ Why everything lives under `~/audiobookshelf/`

The whole stack is **self-contained** in one folder: the `compose.yaml`, the app's `config/` and `metadata/`, and the content itself under `media/`. Because all the volume paths are relative (`./media/Audiobooks`, `./config`), you can move or back up the entire homelab service by copying that one directory. When you add a **Podcasts** library later, it's just another folder, `~/audiobookshelf/media/Podcasts`, mounted at `/podcasts`. (Audiobookshelf is for *spoken* audio; music gets its own server, see the note below.)

What the key settings do:

- `restart: unless-stopped` auto-starts the container on boot and after crashes, exactly what you want on an always-on Pi.
- `ports: 13378:80` maps host port 13378 to the container's port 80 (we avoid host port 80 to leave it free for the reverse proxy in Chapter 4).
- The `volumes` expose your audio under `media/` to the container and persist the database (listening positions, users, settings) next to the compose file, so recreating the container loses nothing. The paths are relative to `~/audiobookshelf/`.

Verify it's running:

```
docker compose ps
curl -I http://localhost:13378          # expect HTTP/1.1 200 OK
```

2.4 Part C: Connect and use it

2.4.1 Step 5: Initial setup in the browser

From any browser on your tailnet, open `http://homelab:13378`. The first-run wizard creates the root user, then drops you at an empty dashboard.

2.4.1.1 5a. Create the root user

This is your admin account. Pick a strong password, Audiobookshelf has no password-reset flow; if you forget it, recovery means editing the SQLite database in `/config` by hand.

2.4.1.2 5b. Add the primary library

Settings (gear) → Libraries → Add Library:

- **Media type: Books**. (This changes the *data model*: Books track one resume position per book in seconds; Podcasts track per-episode played booleans. You want the first for language lessons.)
- **Library name:** `My Course` (specific) or `Audiobooks` (ages better if you'll add more courses). Pick one style and stick to it.

- **Metadata providers:** leave **Open Library** enabled. Even for a homemade rip of a published course it found a clean match for me (cover art and title), where the audiobook-store providers mostly draw blanks on non-commercial audio. You can uncheck the rest to keep scans quick. (For real published audiobooks later, Audible and Google Books are good additions.)
- **Folder:** browse to `/audiobooks`, the *container* path, not the host path. Inside Docker, `~/audiobookshelf/media/Audiobooks` is mounted at `/audiobooks` (from the `volumes:` mapping in the `compose.yaml` in Step 4). If you see `/home/you/Audiobooks` in the picker instead, something's wrong with the volume mount.

Save. Audiobookshelf scans and creates one audiobook with 100 chapters (`L001-LESSON.mp3` … `L100-LESSON.mp3`) in about 30 seconds.

2.4.1.3 5c. Verify the scan

Open the library, open *My Course*: you should see 100 numbered chapters. Click chapter 1. It should play in the browser. If it does, the server is fully functional.

2.4.2 Step 6: Connect the iPhone

1. Install **Audiobookshelf** from the App Store (by `advplyr`, same as the server). Tailscale is already installed and signed in from Chapter 1.
2. Open it, tap **Add Server**:
   - **Server address:** `http://homelab:13378` (or `http://100.x.y.z:13378` using the Pi's Tailscale IP).
   - **Username / password:** the root user from Step 5a.
3. Tap your library → your audiobook → the first track. Audio should stream within a second or two, on Wi-Fi or cellular, anywhere.

2.4.3 Step 7: Daily use

- **Playback speed:** 0.5×–3.0×. 0.85× is great for shadowing dialogue you don't fully understand; 1.25× for review passes. (Enable "Remember playback speed per book" in iOS settings, or it resets to 1.0× each track.)
- **Skip & scrub:** ±10s / ±30s buttons; tap-and-hold to scrub.
- **Bookmarks:** drop a named marker at the current position for phrases to revisit.
- **Cross-device progress:** pause on the iPhone, open the web UI at home, resume at the same second.
- **Offline downloads:** tap the download icon to cache a book locally, your saving grace if a network is hostile to VPNs. Downloaded audio plays with no network at all.

2.5 Maintenance

Update Audiobookshelf (on the Pi):

```
cd ~/audiobookshelf
docker compose pull && docker compose up -d
```

Because config and metadata live in `~/audiobookshelf/`, recreating the container preserves everything.

Back up listening progress with a cron job on the Pi:

```
0 3 * * * tar czf ~/backups/abs-$(date +\%F).tar.gz ~/audiobookshelf/config
~/audiobookshelf/metadata
```

Add more material: drop another folder under `~/audiobookshelf/media/Audiobooks/` (e.g. a Pimsleur course) and click "Scan Library." Each top-level folder becomes its own audiobook.

2.6 Troubleshooting

The iPhone app says "Cannot connect." From the phone's Safari, visit `http://homelab:13378`. If Safari also fails, Tailscale isn't up, open the Tailscale app and confirm both devices show green dots. If Safari works but the app fails, you typed the wrong port (13378).

Tracks play in the wrong order. Audiobookshelf sorts by filename. Rename so they sort correctly (`Lesson 01.mp3`, not `Lesson 1.mp3`), then "Re-scan" in the web UI.

Container won't restart after a reboot. `cd ~/audiobookshelf && docker compose ps`; if it's down, `docker compose up -d`. If `restart: unless-stopped` isn't taking effect, check `systemctl status docker`, Docker itself may not be enabled at boot.

`rsync` reports "Permission denied" on every `mkdir`. Check the destination mode: `ls -ld /path`. A `dr-x------` directory is missing its write bit; fix with `chmod u+w /path` (or `chmod 755`) and re-run.

Work Wi-Fi blocks Tailscale. Tailscale falls back to DERP relays automatically (slower but works). Last resort: download lessons to the iPhone before leaving home, cached audio plays fully offline.

2.7 Recap

- **Copied** the audio off the disc straight onto the Pi with `rsync --no-perms`.
- **Ran** Audiobookshelf on the Pi as a small `docker compose` project.
- **Connected** the iPhone app to `http://homelab:13378`.

Your course is now permanently available from any device on your tailnet, with proper progress tracking and no subscription. Next, in Chapter 3, we give the Pi a second job: blocking ads for every device on your home network with Pi-hole.

3 Chapter 3. Network-wide ad blocking at home with Pi-hole

> **The payoff of this chapter:** every device on your home network, laptops, phones, the smart TV, game consoles, IoT gadgets, has ads and trackers stripped out *before they ever load*, with zero per-device configuration.

Across Chapters 1–2 you put an always-on Raspberry Pi (`homelab`) on your tailnet and gave it its first job, Audiobookshelf. Pi-hole is the natural second tenant: it's a **DNS sinkhole**, a DNS server that answers "no such host" for domains known to serve ads, trackers, and malware, and forwards everything else to a real upstream resolver. Because it works at the DNS layer, it blocks ads in *every* app and browser at once, including places a browser extension can't reach (smart TVs, mobile apps, in-game ads).

This part is entirely about your **home network**. Pi-hole becomes the DNS server for your house, set once at the router so every device on your Wi-Fi inherits it automatically, no per-device tweaking, no remote access, no VPN to think about. Just clean DNS for everything under your roof.

3.1 What you are building (and why this design)

The plan has two halves:

1. **Run Pi-hole on the Pi** (in Docker, alongside Audiobookshelf).
2. **Point your home network at it** by setting Pi-hole as the DNS server in your router, so every device that joins your Wi-Fi automatically uses it.

That router-level step is what makes this effortless: you configure one setting in one place, and every current and future device on your home network is covered without touching any of them individually.

3.2 Part A: Run Pi-hole on the Pi

3.2.1 Step 1: Free up port 53 on the Pi

This is the one host-level snag, and it bites almost everyone, so we do it first. Pi-hole needs to listen on **port 53** (the DNS port). But Raspberry Pi OS (like Debian/Ubuntu) ships `systemd-resolved`, which runs a small "stub" DNS listener bound to `127.0.0.53:53`. Two programs can't own the same port, so Pi-hole will fail to start until you evict the stub.

The surgical fix is to disable *only* the stub listener while leaving `systemd-resolved` running to handle the Pi's own name resolution:

```
# 1. Disable the stub listener via a drop-in (survives package upgrades)
sudo mkdir -p /etc/systemd/resolved.conf.d
printf '[Resolve]\nDNSStubListener=no\n' \
  | sudo tee /etc/systemd/resolved.conf.d/no-stub.conf

# 2. Point the Pi's own resolver at resolved's real file, not the dead stub
sudo ln -sf /run/systemd/resolve/resolv.conf /etc/resolv.conf

# 3. Apply the change
sudo systemctl restart systemd-resolved

# 4. Confirm port 53 is now free (this should print nothing)
sudo ss -tulpn | grep ':53'
```

> Why a drop-in file instead of editing the main config
>
> Dropping a file into `/etc/systemd/resolved.conf.d/` rather than editing the main `resolved.conf` keeps your change isolated and reversible: undoing it is `sudo rm /etc/systemd/resolved.conf.d/no-stub.conf` followed by a restart, with no risk of clobbering a setting a future package update wants to change in the main `resolved.conf`.

If step 4 prints a line mentioning `systemd-resolve`, the stub is still bound, re-check that the drop-in saved correctly and that you restarted the service.

3.2.2 Step 2: Run Pi-hole in Docker

You already have a Docker workflow from Chapter 1, so we'll keep Pi-hole in the same world rather than installing it natively. That means one lifecycle to learn (`docker ...`), declarative config you can back up as a file, and no new system packages on the host.

We'll keep this to one clean, self-contained Compose project: a single folder holding one `compose.yaml`, one `.env` for your settings and secrets, and one `.gitignore`. You can read it, back it up, and version-control it without ever exposing a password.

1. Make the folder.

```
mkdir -p ~/pihole
cd ~/pihole
```

2. Create the `.env`. This is the only place your timezone, Pi IP, and admin password live:

```
cat > .env <<'EOF'
TZ=America/Denver
```

```
PIHOLE_PASSWORD=replace-this-with-a-strong-password
PIHOLE_IP=192.168.1.50
EOF
chmod 600 .env      # readable only by you
```

Set `TZ` to your timezone and `PIHOLE_IP` to your Pi's real LAN address (run `hostname -I` to find it). For the password, type a strong one, or generate one and drop it straight in:

```
# optional: replace the placeholder with a random password
sed -i "s|^PIHOLE_PASSWORD=.*|PIHOLE_PASSWORD=$(openssl rand -base64 18)|"
.env
```

> **Why `PIHOLE_IP`, and why bind the ports to it**
>
> The compose file below publishes Pi-hole's ports on `${PIHOLE_IP}` specifically (e.g. `192.168.1.50:53`) rather than on every interface. Serving on the one LAN address you actually use is tidier and a little safer, Pi-hole listens where it needs to and nowhere else. Make `PIHOLE_IP` the **static or DHCP-reserved** address you assign the Pi in Step 4, so it never changes underneath you.

3. Create `compose.yaml`.

```
cat > compose.yaml <<'EOF'
services:
  pihole:
    container_name: pihole
    image: pihole/pihole:latest

    ports:
      - "${PIHOLE_IP}:53:53/tcp"
      - "${PIHOLE_IP}:53:53/udp"
      - "${PIHOLE_IP}:80:80/tcp"

    environment:
      TZ: "${TZ}"
      FTLCONF_webserver_api_password: "${PIHOLE_PASSWORD:?set
PIHOLE_PASSWORD in .env}"
      FTLCONF_dns_upstreams: "1.1.1.1;1.0.0.1"
      FTLCONF_dns_listeningMode: "ALL"

    volumes:
      - ./etc-pihole:/etc/pihole

    restart: unless-stopped
EOF
```

There is **no password in this file**, only a `${PIHOLE_PASSWORD}` reference that Compose fills in from `.env` at startup. The `${PIHOLE_PASSWORD:?…}` form makes Compose stop with a clear error if that variable is missing, so you can't accidentally launch with a blank password.

4. **Create the `.gitignore`** so the secret and the runtime data can never be committed:

```
cat > .gitignore <<'EOF'
.env
etc-pihole/
EOF
```

> **!** The pattern, in one line
>
> **If it's a password, token, or key, it goes in `.env`, and `.env` goes in `.gitignore`.** The `compose.yaml` holds only `${VARIABLE}` references; the real values live in `.env`, which never leaves your Pi. Every later service in this series uses the exact same pattern.

5. **Start it.**

```
docker compose up -d
docker compose logs --tail 20      # watch it come up
```

> **⚠** Port 80: make sure it's actually free
>
> Audiobookshelf from Chapter 2 publishes host port **13378**, so it does *not* conflict with Pi-hole's port 80. But if you've added anything else that grabs host port 80, Pi-hole's web UI won't start. Check with `sudo ss -tlpn | grep ':80'` before `docker compose up`. (In Chapter 4 we put a reverse proxy on port 80 to give every service a clean URL, at that point Pi-hole's web UI moves to a different port, and we'll handle it there.)

3.2.3 What the key settings do

- **`FTLCONF_*` environment variables** are Pi-hole v6's configuration mechanism. Each one maps to a key in Pi-hole's single config file (`/etc/pihole/pihole.toml`): `FTLCONF_<section>_<key>` → `[section] key`. Anything you set this way becomes **read-only in the web UI**, it's re-applied from the environment on every container start, which is exactly what you want for reproducible, file-defined config.
- **`FTLCONF_webserver_api_password`** is the v6 replacement for the old `WEBPASSWORD` variable. If you omit it, Pi-hole generates a random password and prints it to the container log on first boot.
- **`FTLCONF_dns_upstreams`** is who Pi-hole asks when a domain *isn't* blocked. Cloudflare (`1.1.1.1`) is fast; Quad9 (`9.9.9.9`) adds its own malware filtering. Pick one.
- **`FTLCONF_dns_listeningMode: "ALL"`** tells Pi-hole to answer queries arriving on any interface. In Docker's bridge network, your LAN clients' queries reach Pi-hole *through* the Docker gateway rather than appearing to come from your local subnet, so the stricter "local only" mode would drop them. `ALL` is the documented working choice for a Dockerized Pi-hole.

Pi-hole v6 is meaningfully different from the v5 you'll find in old guides

Pi-hole **v6** (early 2025) collapsed several moving parts into one. There's no more `lighttpd` web server and no PHP, the `pihole-FTL` process now serves the DNS resolver, the web dashboard, *and* a REST API itself. Configuration moved from a pile of files (`setupVars.conf`, `pihole-FTL.conf`, dnsmasq snippets) to a single `/etc/pihole/pihole.toml`. The commands changed too: `whitelist`/`blacklist` became `pihole allow`/`pihole deny`, and the password command is now `pihole setpassword`. If a tutorial tells you to edit `setupVars.conf`, it predates v6, ignore it.

3.2.4 Step 3: Set the admin password and log in

The web UI lives at `http://<pi-ip>/admin`. On your home network use the Pi's LAN IP (e.g. `http://192.168.1.50/admin`); find it with `hostname -I` on the Pi.

Because the password comes from `PIHOLE_PASSWORD` in your `.env`, it's already configured, log in with whatever value you put there. To change it later, edit `.env` and recreate the container:

```
# edit ~/pihole/.env, then:
cd ~/pihole && docker compose up -d --force-recreate
```

Don't fight your own `.env`

Because the password is pinned by the `FTLCONF_webserver_api_password` environment variable, running `pihole setpassword` inside the container won't stick, the value from `.env` is re-applied on every restart and silently wins. With this setup, the `.env` file is the single source of truth for the password; change it there, nowhere else.

3.3 Part B: Point your home network at it

3.3.1 Step 4: Point your home network at Pi-hole

This is the centerpiece. You want every device in the house to use Pi-hole as its DNS server without configuring each one. The cleanest way is to set it **once at your router**, because the router hands out DNS settings to every device via DHCP when they join the Wi-Fi.

1. Find the Pi's LAN IP and lock it down. Run `hostname -I` on the Pi to get the address (e.g. `192.168.1.50`). In your router's admin page, assign that IP as a **DHCP reservation / static lease** for the Pi, so it never changes out from under you. (A Pi-hole whose IP drifts is a house-wide DNS outage.)
2. In the router's settings, find the **DNS server** field, usually under *DHCP*, *LAN*, or *Internet* settings, and set the **primary DNS** to the Pi's IP (`192.168.1.50`).
3. **Leave the secondary DNS blank if you can.** This is counterintuitive: a secondary DNS (like `8.8.8.8`) feels like a safety net, but devices freely use *either* server, so half your queries

would skip Pi-hole and you'd see ads "randomly." A single DNS entry forces everything through Pi-hole.
4. Renew leases so devices pick up the change: reboot the router, or toggle Wi-Fi off/on on each device (or just wait, leases renew on their own).

> **i** What if your ISP router won't let you change DNS?
>
> Some locked-down ISP gateways don't expose the DNS field. Two fallbacks: (a) set DNS **per-device** (each phone/laptop's Wi-Fi settings let you set a manual DNS, more tedious but works), or (b) let Pi-hole run your network's **DHCP** instead of the router (an option in Pi-hole's settings; you'd disable the router's DHCP first). The router approach above is by far the simplest when it's available.

> **⚠** Don't expose port 53 to the internet
>
> Whatever you do, never port-forward port 53 from your router to the Pi. An open, internet-facing DNS resolver gets abused for DNS-amplification attacks within hours. Pi-hole answering your *LAN* is exactly right; Pi-hole answering the *whole internet* is a problem. Keep it on your home network.

3.4 Part C: Add blocklists and verify

3.4.1 Step 5: Add good blocklists

A fresh Pi-hole v6 already subscribes to **StevenBlack's unified hosts list**, which is a strong baseline on its own. You can verify it under **Lists** in the UI. To add more coverage:

1. In the web UI, go to **Lists** (sometimes shown as "Adlists" or "Subscribed Lists").
2. Paste a list URL and click **Add**.
3. Crucially, go to **Tools** → **Update Gravity** (or run `docker exec pihole pihole -g`). Blocklists do *nothing* until "gravity", Pi-hole's compiled blocklist database, is rebuilt.

Two well-maintained additions that block aggressively without breaking sites. Each URL is in its own block below so it copies cleanly, paste one at a time into **Lists** → **Add**:

OISD Big, a popular all-in-one list that's deliberately conservative about breaking sites:

```
https://big.oisd.nl
```

HaGeZi Multi PRO, actively maintained and tiered ("PRO" is the balanced middle tier). The raw URL is long; copy the whole single line:

```
https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/pro.txt
```

> ⚠ Don't stack a dozen lists
>
> More lists is not more better. Two or three quality lists (StevenBlack + OISD *or* HaGeZi) outperform a sprawling pile of overlapping ones and cause far fewer "this website is broken" surprises. Every time you add or remove a list, run **Update Gravity** or nothing changes.

3.4.2 Step 6: Verify the whole thing

Run through these in order; each isolates a different layer:

```
# 1. Pi-hole is up and answering DNS locally on the Pi
dig +short google.com @127.0.0.1            # returns IPs = resolver works
dig +short doubleclick.net @127.0.0.1       # returns 0.0.0.0 = blocking
works

# 2. Pi-hole answers other devices on the LAN (run from your laptop at home)
dig +short doubleclick.net @192.168.1.50    # 0.0.0.0 = LAN clients can use
it
```

Then the real-world test: on a device connected to your home Wi-Fi, open a normally ad-heavy site or app, then watch the **Query Log** at `http://192.168.1.50/admin` from your laptop. You should see that device's LAN IP making queries, with ad domains marked blocked.

If a device's queries don't appear at all, it's still using its old DNS, renew its DHCP lease (toggle Wi-Fi off/on) so it picks up the router's new DNS setting.

3.5 Troubleshooting

Pi-hole container won't start / "port 53 already in use." The `systemd-resolved` stub is still bound. Re-run Step 1 and verify with `sudo ss -tulpn | grep ':53'`. You should see only Docker/Pi-hole, never `systemd-resolve`.

Some devices still show ads after the router DNS change. Either they haven't renewed their DHCP lease (toggle Wi-Fi off/on), or your router has a *secondary* DNS set that lets them bypass Pi-hole. Clear the secondary entry (Step 4.3).

A device hard-codes its own DNS and ignores the router. Some devices (notably certain Android phones, Chromecasts, and anything using DNS-over-HTTPS to `8.8.8.8`) bypass your router's DNS entirely. You can't fix those from the router; either change DNS on the device itself or block their hard-coded resolvers, an advanced topic, but worth knowing it's *them*, not a broken Pi-hole.

A specific site is broken. A blocklist is over-matching. Find the blocked domain in the **Query Log**, click it, and **Allow** it. Then the site works while everything else stays blocked.

You lost the admin password. It's in your `.env` file, `cat ~/pihole/.env` and read `PIHOLE_PASSWORD`. (This is the upside of keeping it in `.env`: it's recoverable from a file only you can read, not lost in a container.)

The Pi itself can't resolve names after Step 1. Check that `/etc/resolv.conf` is the symlink from Step 1.2 and that your upstream (`FTLCONF_dns_upstreams`) is a real public resolver, not the router, if the router points back at Pi-hole (that's a resolution loop).

3.6 Recap

- **Free port 53** by disabling the `systemd-resolved` stub listener.
- **Run Pi-hole** in Docker Compose next to Audiobookshelf.
- **Point your home network at it** by setting the Pi as the router's DNS server (single entry, no secondary), one setting covers every device.
- **Add one or two quality blocklists** and run **Update Gravity**.

Your homelab now does two jobs on your home network. In Chapter 4 we stop typing IP addresses and ports to reach these services, a small reverse proxy plus local DNS gives Pi-hole and Audiobookshelf clean names like **`https://pihole.home`** and **`https://abs.home`** that work from anywhere on your tailnet.

4 Chapter 4. Pretty URLs: a reverse proxy + local DNS

> **The payoff of this chapter:** type a bare `pihole.home` or `abs.home` in any browser on your tailnet (at home or on cellular) and land on the right service over HTTPS, no port numbers, no IP addresses, no scheme to type. And a pattern you'll reuse for every service you add from here on.

Right now you reach your two services awkwardly: `http://192.168.1.50/admin` for Pi-hole, `http://homelab:13378` for Audiobookshelf, an IP here, a port there. This part introduces the machinery that gives **every** service one clean name, starting with these two. When we add the dashboard in Chapter 5, it'll slot into the same system in three lines.

> **!** Why not `pihole.com`?
>
> A `.com` (or any public) name belongs to whoever registered it on the internet, you can't point `pihole.com` at your Pi any more than you can point `google.com` at it. We use a **private** suffix, `.home`, that exists only on *your* network.

4.1 How this works: two jobs, two tools

A working URL needs two separate things:

1. **The name has to resolve to your Pi.** `pihole.home` must turn into the Pi's address, that's **DNS** (we add records to Pi-hole and let Tailscale carry them to every device).
2. **Traffic hitting the Pi has to reach the right service.** `pihole.home` and `abs.home` both arrive on the Pi's web ports (80/443), but Pi-hole and Audiobookshelf listen on *different* ports. Something must read the requested name and forward to the correct backend, a **reverse proxy** (we'll use **Caddy**, the simplest to configure). Caddy also gives each name an HTTPS certificate from its own private CA, so the URLs get a real padlock.

```
pihole.home   DNS   the Pi (443)   Caddy routes by name   pihole:80
abs.home      DNS   the Pi (443)   Caddy routes by name   audiobookshelf:80
```

4.2 Part A: Wire the services onto one network

4.2.1 Step 1: Create the shared Docker network and put Audiobookshelf on it

Caddy reaches each backend by **container name** (`pihole`, `audiobookshelf`), which only works if they share a Docker network. Create it:

```
docker network create homelab
```

Both Audiobookshelf (Chapter 2) and Pi-hole (Chapter 3) are **Compose projects**, so we attach each to the network the *declarative* way, by adding it to the project's `compose.yaml` (see the callout for why that's the only attachment that sticks). Start with Audiobookshelf: add the `networks` blocks to `~/audiobookshelf/compose.yaml`:

```
cat > ~/audiobookshelf/compose.yaml <<'EOF'
services:
  audiobookshelf:
    container_name: audiobookshelf
    image: ghcr.io/advplyr/audiobookshelf:latest

    ports:
      - "13378:80"

    volumes:
      - ./media/Audiobooks:/audiobooks
      - ./config:/config
      - ./metadata:/metadata

    networks:
      - homelab            # NEW: join the shared proxy network

    restart: unless-stopped

networks:
  homelab:
    external: true         # NEW: the shared network created above
EOF
( cd ~/audiobookshelf && docker compose up -d --force-recreate )
```

Any container on the `homelab` network can now reach the others by name, no IP addresses needed between containers.

⚠ Attach Compose services in the Compose file, not with `docker network connect`

It's tempting to run `docker network connect homelab audiobookshelf` instead, but it won't survive. Any time you recreate the container (`docker compose up --force-recreate`, or an update), **a recreate rebuilds it with only the networks declared in its**

compose.yaml, a network you attached by hand is silently dropped, and Caddy would then fail to resolve the backend. So for every Compose service we add the `homelab` network *inside* the Compose file, where a recreate always re-attaches it. Pi-hole gets the same treatment in Step 2.

4.2.2 Step 2: Free port 80 and extend Pi-hole onto the tailnet

Three changes to Pi-hole's `~/pihole/compose.yaml`, all needed before the proxy works:

- **Free host port 80.** Caddy will be the single front door on port 80, so move Pi-hole's web UI to 8081.
- **Listen on the tailnet, not just the LAN.** In Chapter 3 you bound Pi-hole to your LAN IP because it was a home-only service. To make `.home` names (and Pi-hole's own ad-blocking) work *everywhere*, Pi-hole's DNS must also answer on the Pi's Tailscale interface, so drop the `${PIHOLE_IP}:` prefix and let it listen on all interfaces.
- **Join the `homelab` network declaratively** (per the Step 1 callout) so Caddy can reach it as `pihole:80`, and so the recreate below, and every future `docker compose up`, re-attaches it automatically.

Replace `~/pihole/compose.yaml` with this updated version (the `ports:` and new `networks:` blocks are the only changes from Chapter 3):

```
cat > ~/pihole/compose.yaml <<'EOF'
services:
  pihole:
    container_name: pihole
    image: pihole/pihole:latest

    ports:
      - "53:53/tcp"         # all interfaces (LAN + tailnet); was
${PIHOLE_IP}:53:53
      - "53:53/udp"
      - "8081:80/tcp"       # web UI moved off 80, freeing it for Caddy;
was ${PIHOLE_IP}:80:80

    environment:
      TZ: "${TZ}"
      FTLCONF_webserver_api_password: "${PIHOLE_PASSWORD:?set
PIHOLE_PASSWORD in .env}"
      FTLCONF_dns_upstreams: "1.1.1.1;1.0.0.1"
      FTLCONF_dns_listeningMode: "ALL"

    volumes:
      - ./etc-pihole:/etc/pihole

    networks:
```

```
      - homelab            # NEW: declarative attachment, survives every
recreate

    restart: unless-stopped

networks:
  homelab:
    external: true         # NEW: the shared network you created in Step 1
EOF
```

Then recreate the container so the new ports and network take effect:

```
cd ~/pihole && docker compose up -d --force-recreate
```

> **Is listening on all interfaces safe?**
>
> Yes, in this setup. The only networks that can reach the Pi are your home LAN (behind the router) and your tailnet (authenticated WireGuard). The one unbreakable rule, same as Chapter 3: **never port-forward port 53 (or 80) from your router.** As long as you don't, "all interfaces" just means "my LAN and my tailnet," which is exactly who should be able to use it.

Pi-hole's admin UI now lives at `http://192.168.1.50:8081/admin` (your Pi's LAN IP), but in a moment you'll reach it as `https://pihole.home` instead.

4.3 Part B: Deploy the proxy and DNS

4.3.1 Step 3: Deploy Caddy

Caddy is its own small stack:

```
mkdir -p ~/proxy && cd ~/proxy
```

Create `~/proxy/Caddyfile`, the entire routing table:

```
cat > Caddyfile <<'EOF'
# `tls internal` makes Caddy issue each .home name a certificate from its
OWN
# built-in certificate authority (no public CA issues certs for a private
TLD).
# Caddy then serves HTTPS on 443 and auto-redirects http:// -> https://.
Once you
# trust Caddy's local CA on your devices (Step 7), a bare `pihole.home`
typed in
```

```
# the browser just works: no scheme, no warning, real padlock.

pihole.home {
    tls internal
    redir / /admin 302          # land on the admin page directly
    reverse_proxy pihole:80
}

abs.home {
    tls internal
    reverse_proxy audiobookshelf:80
}
EOF
```

Create `~/proxy/compose.yaml`:

```
cat > compose.yaml <<'EOF'
services:
  caddy:
    container_name: caddy
    image: caddy:latest
    ports:
      - "80:80"                 # the single front door (redirects to 443)
      - "443:443"               # HTTPS, on all interfaces (LAN + tailnet)
      - "443:443/udp"           # HTTP/3
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    networks:
      - homelab
    restart: unless-stopped

networks:
  homelab:
    external: true              # the shared network from Step 1

volumes:
  caddy_data:
  caddy_config:
EOF
```

Start it:

```
docker compose up -d
docker compose logs --tail 20
```

Caddy listens on `0.0.0.0:80` and `0.0.0.0:443`, all interfaces, including the Pi's Tailscale `100.x.y.z`, and reaches each backend by container name over the `homelab` network, so it ignores host port mappings entirely. That's why moving Pi-hole's *host* web port to 8081 doesn't bother Caddy: it talks to `pihole:80` inside the network. On first start, `tls internal` makes Caddy generate a local certificate authority and sign a cert for each `.home` name; you'll trust that CA on your devices in Step 7.

4.3.2 Step 4: Add the `.home` names to Pi-hole's DNS

Make the names resolve to the Pi. In the Pi-hole admin UI (`http://192.168.1.50:8081/admin` for now), go to **Settings** → **Local DNS Records** and add one **A record** per name, all pointing at the Pi's **Tailscale** IP (`tailscale ip -4` on the Pi):

Domain	IP
`pihole.home`	`100.x.y.z`
`abs.home`	`100.x.y.z`

Why the *Tailscale* IP, not the LAN IP

A DNS record holds one address. Pointing these at the Pi's `100.x.y.z` tailnet address makes them reachable from *anywhere* a device is on your tailnet, home Wi-Fi or cellular alike. Caddy is listening on that address (Step 3), so the traffic lands. (This assumes Tailscale is running on the device, which is the premise of the whole series.)

4.3.3 Step 5: Make Pi-hole your tailnet's DNS

For `*.home` to resolve on every device, your devices must ask **Pi-hole** for DNS. In the Tailscale admin console (https://login.tailscale.com), **DNS** page:

1. **Add nameserver** → **Custom**, enter the Pi's Tailscale IP (`100.x.y.z`), save.
2. Turn on **Override local DNS**. Now every tailnet device resolves through Pi-hole, so `*.home` names work, *and* you get Pi-hole ad-blocking on every device, even on cellular.

Want only `.home` through Pi-hole, not all your DNS?

Use **split DNS** instead: in **Add nameserver** → **Custom**, enable **Restrict to domain** and set the domain to `home`. Then only `*.home` lookups go to Pi-hole and everything else uses each device's normal DNS. You lose tailnet-wide ad-blocking but keep the pretty URLs. (Chapter 8 discusses how this DNS choice interacts with VPNs when you're away from home.)

4.4 Part C: Use it and trust the cert

4.4.1 Step 6: Try it

From your laptop or phone, anywhere on the tailnet:

```
https://pihole.home    # Pi-hole admin (Caddy redirects to /admin)
https://abs.home       # Audiobookshelf
```

No ports, no IPs. To confirm resolution, `ping pihole.home` should answer from the Pi's `100.x.y.z`. Until you do Step 7, your browser will warn that the certificate isn't trusted (it's Caddy's private CA), that's expected, and Step 7 removes the warning for good.

4.4.2 Step 7: Trust Caddy's certificate (so bare names *just work*)

Without this step, typing a bare `pihole.home` is frustrating: browsers try **`https://` first**, and because they don't yet trust Caddy's private CA, they throw a "not secure" warning (or fall back to treating the name as a web search). The fix is to install **Caddy's root certificate** on each device, once. After that, the browser's automatic HTTPS upgrade lands on a *trusted* cert and bare `home.home` / `pihole.home` / `abs.home` open instantly, padlock and all.

First, export the root certificate from the Caddy container (on the Pi):

```
docker exec caddy cat /data/caddy/pki/authorities/local/root.crt \
  > ~/proxy/caddy-root-ca.crt
```

Now get that file onto each device. Since every device is already on your tailnet, the tidiest way is **Taildrop**, Tailscale's built-in file send between your own machines, no AirDrop, no email, no cable:

```
# from the Pi (or any machine that has the file), send to a peer by its
name:
tailscale file cp ~/proxy/caddy-root-ca.crt <laptop-name>:
tailscale file cp ~/proxy/caddy-root-ca.crt <iphone-name>:
```

Find the exact peer names with `tailscale status`. (If the Pi answers `Access denied: file access denied`, run `sudo tailscale set --operator=$USER` once so the Tailscale CLI works without `sudo`, then retry, or just send from a machine where it already works.)

Then install `caddy-root-ca.crt` on each device:

- **Linux laptop (system-wide):** receive the Taildrop file (it lands in your Downloads, or run `tailscale file get .`), then:

```
sudo cp caddy-root-ca.crt
/usr/local/share/ca-certificates/caddy-root-ca.crt
sudo update-ca-certificates
```

Firefox keeps its **own** trust store: Settings → *Privacy & Security* → *Certificates* → **View Certificates** → *Authorities* → **Import** the file and tick "Trust this CA to identify websites."

- **iPhone:** open the **Tailscale** app, tap the received `caddy-root-ca.crt`, and **Save to Files**. Open it from the **Files** app → iOS says "Profile Downloaded" → **Settings → General → VPN & Device Management → Install** the profile. Then (this second step is the one people miss) **Settings → General → About → Certificate Trust Settings** and toggle **on** "Enable Full Trust" for the Caddy Local Authority. (AirDrop or email work too, but Taildrop keeps it on the tailnet.)

Now reload `https://home.home`, the warning is gone, and typing the bare name works.

> **Why a private CA instead of a real certificate**
>
> A real, publicly-trusted certificate requires a domain **you own** and a public CA (Let's Encrypt) that can verify it, which can't be done for a private `.home` name. Caddy's internal CA sidesteps that entirely: it mints certificates locally and you tell *your* devices to trust *that* CA. The trade-off is the one-time install above. (If you ever buy a real domain, point a subdomain at the Pi and drop `tls internal`, Caddy will fetch a real certificate automatically, and you can remove the root cert from your devices.)

4.5 The pattern you'll reuse

Every service from here on gets a pretty URL the same way, three small steps:

1. Put its container on the `homelab` network.
2. Add a block to `~/proxy/Caddyfile`: `name.home { tls internal\n reverse_proxy container:PORT }`.
3. Add a `name.home → 100.x.y.z` record in Pi-hole, then `docker compose restart caddy` (in `~/proxy`).

Because you've already trusted Caddy's CA, the new name gets a trusted cert automatically, no per-service certificate work. Chapter 5 does exactly this for the dashboard, giving it `https://home.home`.

4.6 Caveats

- **One-time CA trust per device.** The padlock only appears on devices where you've installed Caddy's root cert (Step 7). A brand-new device shows the warning until you do. This is the cost of a private TLD; it's a one-time step.

- **Exit nodes break `.home`.** A Mullvad exit node routes DNS through Mullvad and bypasses Pi-hole, so `*.home` won't resolve while one is active. This tension is the subject of Chapter 8; switch the exit node off (or use your home Pi as the exit node) to use the names.

4.7 Recap

- **Shared `homelab` network** so Caddy can reach services by name.
- **Pi-hole** moved its web UI to `8081` and now listens on all interfaces (LAN + tailnet).
- **Caddy** is the single front door on ports 80/443, serving HTTPS with its own internal CA and routing `pihole.home` and `abs.home` to the right containers.
- **Pi-hole DNS records + the Tailscale Override** make those names resolve on every device, everywhere.
- **Trusting Caddy's root CA** on each device makes bare names open over HTTPS with no warning and no scheme.

You now have clean, portable URLs, and a repeatable recipe for every service to come. Next, Chapter 5 adds a dashboard that ties them all together behind `https://home.home`.

5 Chapter 5. A one-URL dashboard at `https://home.home`

> **The payoff of this chapter:** open any browser on your tailnet, type `https://home.home`, and land on a clean dashboard that shows your services at a glance and links into a full Docker management console, the single front door that ties everything together.

Chapter 4 gave individual services clean names (`https://pihole.home`, `https://abs.home`). This part adds the *landing page* that gathers them in one place, plus a real management GUI, and it slots into the reverse proxy you built in Chapter 4 using the exact three-step pattern from the end of that chapter.

We'll install two complementary tools:

Tool	What it does	Why both
Homepage	A fast, configurable dashboard with live "tiles" for each service	The pretty front door, *view* status, click through to everything
Portainer	A full web GUI for Docker	The *manage* half, start/stop/restart containers, read logs, update images, all by clicking

Homepage can *show* container status but can't *control* containers; Portainer *controls* Docker beautifully but isn't a customizable landing page. Together they're the standard beginner homelab combo, and Homepage links to Portainer with a single tile.

> **Why Homepage over CasaOS/Cockpit/Homarr**
>
> **CasaOS** wants to own the whole box and install its own Docker stack. It fights the hand-rolled containers you've built so far. **Cockpit** manages the host OS, not Docker, so it's off-target here. **Homarr** is a great click-to-edit alternative if you hate YAML. Homepage wins for a written guide because its config is plain text you can copy-paste, back up, and reproduce exactly.

5.1 Part A: Deploy Homepage and Portainer

5.1.1 Step 1: Deploy Homepage and Portainer

Make the project folder (one stack for the whole control panel):

```
mkdir -p ~/dashboard && cd ~/dashboard
```

Create `~/dashboard/compose.yaml`. Note Homepage has **no host port**, Caddy (from Chapter 4) is the front door on port 80 and will reach Homepage by container name over the shared `homelab` network:

```
services:
  homepage:
    image: ghcr.io/gethomepage/homepage:latest
    container_name: homepage
    volumes:
      - ./config:/app/config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    environment:
      # Hostnames Caddy will forward here. Comma-separated, NO spaces.
      HOMEPAGE_ALLOWED_HOSTS: home.home,homelab,homelab.your-tailnet.ts.net
      # Widget secrets, injected from .env: never hard-coded in the config
files.
      HOMEPAGE_VAR_PIHOLE_PASSWORD: ${PIHOLE_PASSWORD:?set PIHOLE_PASSWORD
in .env}
      HOMEPAGE_VAR_ABS_TOKEN: ${ABS_TOKEN:-}
    networks:
      - homelab
    restart: unless-stopped

  portainer:
    image: portainer/portainer-ce:lts
    container_name: portainer
    ports:
      - "9443:9443"          # HTTPS management UI (self-signed cert)
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data
    networks:
      - homelab
    restart: always

networks:
  homelab:
    external: true           # the shared network created in Chapter 4

volumes:
  portainer_data:
```

Create this project's `.env` and `.gitignore`, the same pattern as every other stack:

```
cat > ~/dashboard/.env <<'EOF'
PIHOLE_PASSWORD=same-password-you-set-for-pihole
ABS_TOKEN=your-audiobookshelf-api-token
EOF
chmod 600 ~/dashboard/.env

cat > ~/dashboard/.gitignore <<'EOF'
.env
EOF
```

Get the Audiobookshelf token from its web UI under **Settings → Users → your user → API token**, then bring the stack up:

```
cd ~/dashboard && docker compose up -d
```

> Two different "reachability" mechanisms
>
> Homepage uses the **Docker socket** to read container *status* (running/stopped, CPU, RAM), and the **`homelab` network** to call service *APIs* for widget data. The socket gives you the green "running" dot; the network gives you "1,204 ads blocked today."

> Portainer's 5-minute clock
>
> The first time you open Portainer (`https://homelab:9443`, accept the self-signed cert warning once), it asks you to create an admin user. Do it **within a few minutes** of starting the container, or Portainer locks itself and you must `docker restart portainer` to reopen the window. The `:lts` image runs natively on a 64-bit Pi.

5.1.2 Step 2: Configure the tiles

> ! Why your dashboard looks empty at first
>
> Homepage does **not** auto-discover anything, out of the box it creates *blank* config files and shows a nearly empty page. Everything you see is whatever you put in `./config/*.yaml`. The blocks below are a deliberately *full* starting point (service tiles, a bookmarks row, and an info bar) so the page looks alive immediately.

Homepage reads YAML from the `./config` folder. Replace the auto-generated blank files with these.

`config/settings.yaml`, look and section layout:

```yaml
title: My Homelab
theme: dark
```

```
color: slate
headerStyle: boxed
layout:
  Media:
    style: row
    columns: 2
  Network:
    style: row
    columns: 2
  Management:
    style: row
    columns: 2
  Links:                 # a bookmarks group (defined in bookmarks.yaml
below)
    style: row
    columns: 4
```

`config/docker.yaml`, lets tiles show live container status via the socket:

```
my-docker:
  socket: /var/run/docker.sock
```

`config/services.yaml`, the tiles, now pointing at the pretty URLs from Chapter 4:

```
- Media:
    - Audiobookshelf:
        icon: audiobookshelf.png
        href: https://abs.home                  # the pretty URL from
Chapter 4
        description: Audiobooks & language courses
        server: my-docker
        container: audiobookshelf              # -> live status dot +
CPU/RAM
        widget:
          type: audiobookshelf
          url: http://audiobookshelf:80         # container name + internal
port (for widget data)
          key: '{{HOMEPAGE_VAR_ABS_TOKEN}}'     # injected from .env, no
real token in this file

- Network:
    - Pi-hole:
        icon: pi-hole.png
        href: https://pihole.home               # the pretty URL from
Chapter 4
        description: DNS ad-blocking
```

```yaml
        server: my-docker
        container: pihole
        widget:
          type: pihole
          url: http://pihole:80                    # container name +
internal port
          version: 6                               # REQUIRED for Pi-hole v6
(defaults to 5!)
          key: '{{HOMEPAGE_VAR_PIHOLE_PASSWORD}}'  # injected from .env, no
real password in this file

- Management:
    - Portainer:
        icon: portainer.png
        href: https://homelab:9443
        description: Manage Docker containers
```

`config/widgets.yaml`, the info bar across the top:

```yaml
- resources:
    cpu: true
    memory: true
    disk: /
- search:
    provider: duckduckgo
    target: _blank
- datetime:
    format:
      timeStyle: short
- greeting:
    text_size: xl
    text: Welcome to the homelab
```

`config/bookmarks.yaml`, a row of quick links (pure links, no Docker logic):

```yaml
- Links:
    - Tailscale admin:
        - abbr: TS
          href: https://login.tailscale.com/admin/machines
    - Router:
        - abbr: RT
          href: http://192.168.1.1
    - Pi-hole docs:
        - abbr: PH
          href: https://docs.pi-hole.net
    - Audiobookshelf docs:
```

```yaml
      - abbr: AB
        href: https://www.audiobookshelf.org/docs
```

Apply the config (a plain restart re-reads these files):

```bash
cd ~/dashboard && docker compose restart homepage
```

> ⚠ The Pi-hole v6 widget gotcha
>
> If the Pi-hole tile shows "API error," the most common cause is a missing `version: 6` line, the widget defaults to the old v5 API, which no longer exists. The `key` for v6 is your admin password, not a legacy API token.

5.2 Part B: Give it a pretty URL and use it

5.2.1 Step 3: Give the dashboard its pretty URL

This is the Chapter 4 pattern, applied to the dashboard. Add a Caddy route and a Pi-hole record so `https://home.home` (and the alias `https://homelab`) open the dashboard. `tls internal` gives it the same trusted-cert treatment as Chapter 4's names, no extra certificate work.

1. Add a block to `~/proxy/Caddyfile`:

```
home.home, homelab {
    tls internal
    reverse_proxy homepage:3000
}
```

2. Reload Caddy:

```bash
cd ~/proxy && docker compose restart caddy
```

3. Add a Pi-hole Local DNS record (Settings → Local DNS Records): `home.home` → your Pi's Tailscale IP (`100.x.y.z`), exactly like the records you added in Chapter 4.

> ⚠ `HOMEPAGE_ALLOWED_HOSTS` is the #1 "blank page" trap
>
> Homepage refuses to render for any hostname not in its allowed list. Caddy forwards `home.home` and `homelab` to it, so both must appear in `HOMEPAGE_ALLOWED_HOSTS` (they do, in Step 1). If you add another name later, add it there too and `docker compose up -d --force-recreate homepage`. This variable only takes effect on a recreate, not a plain restart.

5.2.2 Step 4: Try it

From your laptop or phone, anywhere on the tailnet:

```
https://home.home      # (or https://homelab, both open the dashboard)
```

You should see your dashboard with live tiles for Audiobookshelf and Pi-hole and a link into Portainer. From the Portainer tile you can start, stop, inspect, and update every container by clicking, including pulling new images.

5.3 Troubleshooting

`https://home.home` shows a blank page. `HOMEPAGE_ALLOWED_HOSTS` doesn't include the host you typed. Add it and `docker compose up -d --force-recreate homepage`.

`https://home.home` times out / "can't connect." Either the name isn't resolving (check the Pi-hole record from Step 3; try `ping home.home`) or Caddy isn't routing it (confirm the block is in the `Caddyfile` and you restarted Caddy). `docker logs caddy` shows routing errors.

Widgets show "API error" or stay blank. The widget can't reach the service. Confirm the container is on the `homelab` network (`docker network inspect homelab`) and that you used the container name in the widget `url`. For Pi-hole, confirm `version: 6` and the correct password.

Portainer won't let me create an admin user. The security timeout elapsed, `docker restart portainer` and reopen `https://homelab:9443` promptly.

5.4 Recap

- **Homepage** (no host port) + **Portainer** (on 9443), both on the `homelab` network, deployed as one `~/dashboard` stack.
- **Tiles** point at the pretty URLs from Chapter 4 and pull live status via the Docker socket and service APIs.
- **Added the dashboard to the proxy**, one Caddy block + one Pi-hole record, so `https://home.home` (and `https://homelab`) open it from anywhere.

Your homelab now has a single, memorable front door. In Chapter 7 we turn to privacy on the *outbound* side: what a VPN does, how Aura and Mullvad compare, and what a Tailscale *exit node* is.

6 Chapter 6. Remoting into your machines from your phone

> **The payoff of this chapter:** open a terminal on your Pi, or a full graphical desktop on your laptop, straight from your phone, anywhere, by typing the machine's name. No IP addresses, no port forwarding, just the Tailscale name.

By now everything on your tailnet is reachable by name. This chapter puts that to work from the device you always have on you: your phone. Two tools cover the two things you'll actually want:

Tool	What it gives you	Good for
Termius	An SSH terminal (command line)	The headless Pi, or any server
NoMachine	A full remote **desktop** (mouse, windows)	A machine that has a GUI (your laptop/desktop)

The Pi has no desktop, so you reach it with **Termius** (SSH). When you want to click around a real desktop from your phone, that's **NoMachine**, pointed at a machine that actually runs one.

> Name your machines first, it makes all of this painless
>
> Everything below addresses machines by their **Tailscale name**, so before anything else, give each device a clear, memorable name. In the Tailscale admin console (https://login.tailscale.com) open the **Machines** page, and for each device use the menu and **Edit machine name** to set something obvious like `homelab`, `laptop`, or `desktop`. With MagicDNS on (Chapter 1), every device on your tailnet can then reach each one by that bare name. Vague auto-generated names like `desktop-3f2a` are the thing that makes remote access annoying; fix that once here.

6.1 Part A. SSH from your phone with Termius

Termius is a free, polished SSH client for iOS and Android. Because your phone runs Tailscale, it resolves your machine names directly, so you connect to `homelab` exactly like you would from your laptop.

6.1.1 Step 1: Confirm your phone is on the tailnet

Open the **Tailscale** app on the phone and make sure it's connected (you set this up in Chapter 1). That's what lets `homelab` resolve. On cellular or any Wi-Fi, it just works.

6.1.2 Step 2: Install Termius and add the Pi as a host

Install **Termius** from the App Store or Play Store, then add a new host:

- **Hostname / Address:** the Tailscale name of the machine, e.g. `homelab` (or its full name `homelab.your-tailnet.ts.net`, or the `100.x.y.z` Tailscale IP if a bare name ever doesn't resolve).
- **Username:** your account on that machine (the one you use for `ssh you@homelab`).
- **Port:** 22.

6.1.3 Step 3: Authenticate with an SSH key

Your Pi only accepts **key-based** login (Chapter 1), so Termius needs a key it trusts. Two easy ways:

- **Generate a key in Termius** (Keychain → new key), copy its **public** key, and append it to the Pi's `~/.ssh/authorized_keys` from a machine that's already logged in:

```
echo 'ssh-ed25519 AAAA... (the public key Termius shows you)' >>
~/.ssh/authorized_keys
```

- **Or import the key you already use** from your laptop into Termius's Keychain, since that key is already authorized on the Pi.

Attach the key to the host, tap to connect, and you have a terminal on the Pi from your phone. From here you can run `docker compose`, check logs, restart a service, anything you'd do over SSH at your desk.

6.2 Part B. A full desktop with NoMachine

Sometimes you want a real **graphical desktop**, not a terminal: a browser, a file manager, a GUI app. **NoMachine** streams a machine's desktop to your phone, fast enough to actually use. It needs a machine that **has** a desktop, so this is for your **Linux laptop or desktop**, not the headless Pi.

6.2.1 Step 4: Install the NoMachine server on the GUI machine

On the machine whose desktop you want to reach (your laptop/desktop), download the NoMachine package for your system from https://www.nomachine.com/download and install it. On Debian/Ubuntu that's the `.deb`:

```
sudo dpkg -i nomachine_*.deb      # then, if it complains about deps:
sudo apt --fix-broken install
```

Installing it starts the NoMachine **server**, which listens on port **4000**. You don't need to open any router ports: your phone reaches it over Tailscale.

> ⚠ Keep NoMachine on the tailnet only
>
> Don't port-forward 4000 from your router. NoMachine should be reachable **only** over your tailnet (and your LAN), exactly like everything else in this series. The Tailscale tunnel is already encrypted; NoMachine adds its own login on top.

6.2.2 Step 5: Connect from the phone by name

Install the **NoMachine** app on your phone, then add a connection:

- **Host:** the machine's Tailscale name, e.g. `desktop` (or its `100.x.y.z`).
- **Port:** 4000.
- **Protocol:** NX.

Log in with that machine's normal user account, and its desktop appears on your phone, controllable by touch. The machine needs to be **on, awake, and logged in to a desktop session** for this to work. When you only need the command line, or you're reaching the Pi, use Termius from Part A instead.

6.3 Recap

- **Name every machine** clearly on the Tailscale Machines page so the rest is just typing a name.
- **Termius** gives you an SSH terminal from your phone, addressed by Tailscale name, with key auth, the right tool for the headless Pi.
- **NoMachine** gives you a full remote desktop from your phone, for machines that actually run a desktop, over the tailnet with nothing exposed to the internet.

Part II

Volume II: On the road and your devices

7 Chapter 7. VPN privacy: Aura, Mullvad, and your options

> **The payoff of this chapter:** understand what a VPN actually does for you, see how the VPN you already use (Aura) compares to a privacy-focused option (Mullvad), and learn the Tailscale-native way to route traffic, the *exit node*, so you can pick the setup that fits. The messier question of how all this behaves *away from home* (and how it collides with your Pi-hole) gets its own treatment in Chapter 8.

Everything so far has been about reaching *into* your homelab. This part is the opposite direction: making your devices' traffic leave through a VPN, so the websites and apps you use can't see your real IP address, and so your ISP or a café's Wi-Fi can't see what you're doing.

You already use **Aura VPN** to change locations. That's a perfectly good consumer VPN. The goal here isn't to talk you out of it, it's to lay out the landscape clearly: what a VPN does, where Aura fits, where **Mullvad** fits, and the one Tailscale concept (the **exit node**) that ties VPNs into the homelab you've been building.

7.1 What a VPN does (and what it doesn't)

A VPN routes all your internet traffic through a remote server before it reaches the wider internet. Two practical consequences:

- **Your IP is hidden.** Sites see the VPN server's address, not yours, useful for privacy and for appearing to be in another location.
- **Your local network can't snoop.** On untrusted Wi-Fi, the airport or hotel can see only that you're talking to a VPN, not what you're doing.

What a VPN does *not* do: it isn't anonymity (the VPN provider can see your traffic, so provider trust matters), and it doesn't block ads by itself unless the provider offers DNS-level filtering. Keep that ad-blocking caveat in mind, it's the seam where VPNs and your Pi-hole rub against each other, which is exactly the Chapter 8 discussion.

7.2 The Tailscale exit node: a VPN built from your own tailnet

Before comparing products, here's the concept that makes Tailscale relevant to VPNs at all.

Normal Tailscale is an **overlay network**: it only carries traffic *between* your own devices (laptop `homelab` phone). Your ordinary internet traffic still leaves through your local connection; Tailscale doesn't touch it.

An **exit node** changes that. You designate one device on your tailnet as "the exit," and other devices route **all** their internet traffic through it, the full default route, not just tailnet traffic. To the outside world, your traffic now appears to come from the exit node's IP. That's exactly how a traditional full-tunnel VPN behaves, except the "VPN server" is a node on your own tailnet. Only one exit node is active per device at a time, and you switch it off by selecting "None."

This matters because Tailscale can use **Mullvad's servers as exit nodes**, giving you a real privacy VPN *inside* the same Tailscale app you already run, with no second VPN client. That's the key to making VPN and homelab coexist, and it's why Mullvad gets special attention below.

7.3 Your three real options

7.3.1 Option A: Tailscale's Mullvad add-on *(best fit for this homelab)*

Tailscale sells access to Mullvad's worldwide WireGuard servers as a paid add-on **purchased through Tailscale**, no separate Mullvad subscription. Mullvad's servers simply appear as selectable exit nodes inside your tailnet.

- **Cost:** about **$5/month**. Tailscale describes this as 5 "licenses," where one license covers up to 5 devices. (Tailscale's marketing page and its docs state the device math slightly differently, confirm the exact count at the checkout screen before buying.)
- **Enable it:** admin console → **Settings** → the **Mullvad** section → **Configure**, complete checkout, then grant your devices access.
- **Use it (Linux):**

  ```
  tailscale exit-node list                      # list Mullvad cities/servers
  tailscale set --exit-node=<mullvad-node> # route everything through it
  tailscale set --exit-node=                    # turn it back off
  ```

  On **iOS / macOS / Android**: the ••• menu → **Use exit node** → pick a Mullvad **location**.
- **Why it's the best fit here:** it *is* Tailscale, so it coexists with everything you've built, and crucially, it's the only option that works on your iPhone *without* fighting Tailscale (more on that in Chapter 8). The `--exit-node` choice is persistent across reboots, so "always on" is just setting it once.

7.3.2 Option B: A standalone VPN app (Aura, or the Mullvad app)

This is what you do today with **Aura**: a self-contained VPN app on each device that you toggle on to change location.

- **Aura:** part of the Aura identity-protection suite. A closed consumer app, easy to use, location-switching, with split tunneling on its supported platforms. It's fine for casual "make me look like I'm elsewhere." Its limitations for *this* project: it's app-only (there's no client you can run on the headless Pi, and no WireGuard config files to reuse), and like any full-device VPN it takes over DNS while it's on.

- **Mullvad app:** a flat **€5/month** (~$5.40), up to 5 devices, includes a proper **kill switch**, and (unlike Aura) Mullvad hands you **WireGuard config files** you can run anywhere, including on the Pi. That openness is what makes the advanced setups in Chapter 8 possible.

The catch with *any* standalone VPN app is how it coexists with Tailscale, a genuinely thorny topic that's the heart of Chapter 8. The short version: **phones run only one VPN at a time**, so the Aura/Mullvad app and Tailscale are mutually exclusive on your iPhone.

7.3.3 Option C: Your Pi as a self-hosted exit node (free, but not private)

You can advertise one of your *own* machines as an exit node:

```
sudo tailscale set --advertise-exit-node
# then approve it: admin console -> Machines -> the device ->
#   Edit route settings -> enable "Use as exit node"
```

On Linux you also enable IP forwarding (`net.ipv4.ip_forward=1` and the IPv6 equivalent).

- **What it's good for:** routing through your **home** Pi so you appear to be at home, reaching home-only services, or content tied to your home connection, and, importantly, it's the one exit option that keeps your traffic on your own network where **Pi-hole still applies**.
- **What it's not:** privacy. Traffic exits from **your own IP**, so there's no hiding. And don't use a *phone* as an exit node. It routes in userspace and is slow; the Pi (kernel routing) is fine.

7.4 Recommendation

For a homelab that's already built on Tailscale and a user who wants privacy *and* wants it to mesh with everything else:

- **For privacy / location changing: Option A (Tailscale's Mullvad add-on).** It's the same price ballpark as Aura or a standalone Mullvad sub, but it lives inside the one VPN your devices already run, so it's the only choice that works cleanly on your iPhone alongside the rest of the homelab, and the setting persists across reboots for true "always on."
- **Keep Option C (Pi as exit node) in your pocket** as a complement, not a replacement: flip to it when you want to appear at home or keep Pi-hole filtering active.
- **Aura (Option B) is fine to keep** as a standalone tool, but understand it won't integrate with the homelab. It can't run on the Pi, and on your phone it can't run at the same time as Tailscale. That trade-off, and how to live with it on the road, is precisely what Chapter 8 unpacks.

> ℹ "Always on" in one line
>
> With Option A, `tailscale set --exit-node=<mullvad-node>` (or picking a location in the mobile app) sticks across reboots and reconnects. You set it once per device. If the exit node ever becomes unreachable, traffic *fails closed* rather than leaking out your real connection, which is the behavior you want for an always-on privacy setup.

7.5 Recap

- A **VPN** hides your IP and shields your traffic from the local network, but it isn't anonymity and doesn't block ads on its own.
- A Tailscale **exit node** is a VPN built from your tailnet; with the **Mullvad add-on**, Mullvad's servers *are* your exit nodes, inside the app you already run.
- Three options: **A)** Tailscale's Mullvad add-on (best fit, works on iPhone), **B)** a standalone app like Aura or Mullvad's own, **C)** your Pi as a free-but-not-private exit node.
- **Recommendation:** use Option A for privacy, keep Option C for appear-at-home, and know Aura's limits before relying on it.

You now know your VPN options and how to turn each one on. The hard part, what actually happens to your Pi-hole, your homelab access, and your VPN when you walk out the front door, and how to get a setup that doesn't fight itself, is next, in Chapter 8.

8 Chapter 8. On the road: your homelab, Pi-hole, and VPN away from home

> **The payoff of this chapter:** a clear, honest map of what you can and can't have when you leave the house, reaching your homelab remotely, carrying Pi-hole's ad-blocking with you, and running a VPN, plus the one rule that explains why you can't always have all three at once on the same device.

At home, everything is easy: your devices share the Pi's network and everything just works. The moment you walk out the door, the only thing connecting your phone to your Pi is **Tailscale**. This part is the field manual for that world.

8.1 Reaching your homelab from anywhere (the easy part)

This one already works, and it's the cleanest win. Because Tailscale is a mesh VPN, any device signed into your tailnet reaches the Pi from anywhere, coffee shop, cellular, hotel, with no extra setup:

- **Audiobookshelf:** `https://abs.home` streams just as it does at home.
- **The dashboard:** `https://home.home` (or `https://homelab`) opens from anywhere.

The reason it's effortless is that you're only reaching *into* your own network over an authenticated tunnel. The hard parts below are all about the *outbound* direction, what your traffic does and which DNS resolves it.

8.2 Pi-hole already follows you (set up in Chapter 4)

You don't need to do anything here, when you made Pi-hole your tailnet's DNS in Chapter 4 (the **Override local DNS** setup), you also got Pi-hole's ad-blocking on every device, everywhere. On cellular, your phone still resolves through Pi-hole, so ads stay blocked. The same step is also what makes your `.home` names work away from home.

That's the good news. The catch is what happens when you turn on *another* VPN, which is the rest of this chapter.

> **!** This only holds while Tailscale is the active VPN
>
> The whole mechanism rides on Tailscale being the thing controlling your device's DNS. The instant another VPN takes over (next section), this stops applying. That's not a bug, it's the

central tension of going mobile, and the rest of this chapter is about navigating it.

8.3 The one rule that governs everything off-network

Here it is, the rule that explains every "why can't I just…" on the road:

> **A full-device VPN owns the default route *and* the DNS. A phone runs only one VPN at a time.**

Two consequences fall out of it:

1. **Aura and Tailscale can't both be on (on a phone).** iOS and Android allow exactly one active VPN. Turn on Aura to change location, and Tailscale drops , which means your Pi-hole-over-Tailscale push and your `https://homelab` access both go dark, because the Pi's `100.x` address is only reachable over Tailscale.
2. **Whatever VPN is active forces its own DNS.** Even setting the one-VPN limit aside, the active VPN routes DNS through its own resolvers. So while you're on *any* location/privacy VPN, your traffic is *not* going through Pi-hole, by design, to prevent DNS leaks.

`Insight` This is why "use my own VPN *and* filter through Pi-hole, on my phone, away from home" has no clean answer: Pi-hole lives only on your tailnet, reaching it needs Tailscale to be the active VPN, and a phone can't run two VPNs. The only ways out are to (a) make Pi-hole publicly reachable (a security no-go) or (b) make the VPN and Pi-hole live on the *same box you route through*, which is the Mullvad-via-Tailscale exit-node architecture, not a separate app like Aura.

8.4 The decision matrix

You can't have all three of {homelab access, Pi-hole filtering, privacy VPN} on one device at one time. But you *can* switch between coherent modes in seconds. Pick per situation:

You're away and you want…	Set this	Pi-hole?	Homelab?	Hides IP?
Homelab + ad-blocking (the everyday default)	Tailscale on, no exit node	via DNS push		(your real IP)
Appear at home / reach LAN	Tailscale + **Pi as exit node** (Chapter 7 C)			(home IP)
Privacy / change location	**Mullvad exit node** (Chapter 7 A)	Mullvad's DNS	(LAN access on)	

You're away and you want...	Set this	Pi-hole?	Homelab?	Hides IP?
Privacy via your existing app	**Aura** on	Aura's DNS	(Tailscale off)	

The pattern to read out of that table:

- **The Mullvad exit node (Option A) is the only privacy choice that keeps your homelab reachable**, because it *is* Tailscale, your tunnel to `homelab` stays up while Mullvad handles egress. It does, however, hand DNS to Mullvad, so Pi-hole is bypassed (use Mullvad's own ad-blocking DNS as a substitute).
- **Aura gives you privacy but takes everything else down** while it's on, since it displaces Tailscale entirely on the phone. Fine for a quick "look like I'm elsewhere" session; not a mode you live in if you want your homelab.
- **The Pi-as-exit-node mode is the sweet spot when you want Pi-hole filtering on the road**, your traffic goes home, gets filtered by Pi-hole, and exits from your home IP. No privacy, but full ad-blocking and full homelab access.

8.5 Making a Mullvad exit node behave on the road

If you go with the Mullvad add-on (Chapter 7's recommendation), two flags matter when you're mobile:

```
# Keep access to LAN devices (printers, etc.) while using an exit node
tailscale set --exit-node=<mullvad-node> --exit-node-allow-lan-access
```

- **`--exit-node-allow-lan-access`**: by default, turning on *any* exit node cuts your access to the local network you're physically on. This flag restores it. On mobile, it's the **Allow LAN access** toggle when you pick the exit node.
- **DNS leaks:** allowing LAN access can let some DNS queries escape the tunnel. Recent Tailscale clients (1.48.3+) route DNS correctly through Mullvad exit nodes without extra config, keep your clients updated.

> ⚠ A real bug to watch for
>
> Some Tailscale client versions around 1.82 had a defect where an **active exit node bypassed the global Pi-hole nameserver**, so your ad-blocking would quietly stop whenever an exit node was on, even when you expected the DNS push to apply. If you see that symptom, update the Tailscale client first. It's a known issue, not your misconfiguration.

8.6 A practical everyday routine

For most people the comfortable default is simple:

1. **Leave Tailscale on, no exit node, all the time.** This is the everyday mode: homelab reachable, Pi-hole ad-blocking everywhere via the DNS push, and your phone behaves normally. You give up IP-hiding, which most of the time you don't need.
2. **When you specifically want privacy** (sketchy Wi-Fi, want to change region): flip on the **Mullvad exit node** for that session. Accept that Pi-hole steps aside for Mullvad's DNS while it's on, then flip it back to None when you're done.
3. **Retire Aura, or keep it only as an occasional standalone tool.** Anything Aura does for you on the road, the Mullvad exit node does too, while staying inside the one VPN that keeps the rest of your homelab alive. The only reason to reach for Aura is if you specifically want its app or its kill switch on a device where you're not using Tailscale anyway.

8.7 Recap: and the whole series in one breath

- **Reaching your homelab away** is the free win: Tailscale makes `homelab:13378` and `https://home.home` work from anywhere, no extra setup.
- **Pi-hole on the road** takes one change, add the Pi as a custom nameserver in the Tailscale admin console and turn on **Override local DNS**, and works only while Tailscale is your active VPN.
- **The governing rule:** a full-device VPN owns DNS, and a phone runs one VPN at a time, so homelab access, Pi-hole filtering, and a privacy VPN can't all be on at once. Switch between modes using the decision matrix.
- **Best privacy-with-homelab combo:** the Mullvad exit node, because it lives inside Tailscale. **Best ad-blocking-on-the-road combo:** the Pi as your exit node. **Aura** stays a standalone fallback.

With that, your homelab is fully mobile: an always-on Raspberry Pi that streams your media, blocks ads on every device, presents a single `https://home.home` control panel, and (when you understand the one-VPN rule) gives you a deliberate, switchable answer for privacy and ad-blocking the moment you step out the door. All of it private by default, over one Tailscale network, with nothing exposed to the public internet.

One direction is still missing: getting your *phone and your Linux machines* to talk to each other for everyday file transfer and clipboard sharing. That's Chapter 9, the finale.

9 Chapter 9. Wiring your iPhone into your Linux machines

> **The payoff of this chapter:** move files between your iPhone and your Linux laptop in one tap, share a clipboard and see your phone's battery on the desktop, do the same to your Pi from anywhere over Tailscale, and know the one wired fallback (USB) for when Wi-Fi isn't an option, without installing a pile of tools you'll never use.

You've built a homelab your iPhone can reach. This part closes the loop in the *other* direction: getting your phone and your Linux machines to talk to each other for the everyday stuff, "send me that PDF," "paste this URL onto my desktop," "pull these photos off my phone." On a Mac you'd reach for AirDrop and Handoff; on Linux you assemble the same conveniences from two small apps, plus one wired option you'll rarely need.

The important framing up front: **these are three different tools solving three different problems.** Don't think "which one wins", think "which job am I doing."

9.1 The two Wi-Fi tools, and why you want both

Job	LocalSend	KDE Connect (on iPhone)
Send files phone → Linux		
Receive files Linux → phone		
Share clipboard		
Phone battery on desktop		
Media remote control		
Mirror phone notifications to desktop		**on iPhone** (Android-only)
SMS / run commands from desktop		**on iPhone** (Android-only)
Dead-simple, one-purpose app		

> ❗ The iPhone reality check (this is where most guides mislead you)
>
> KDE Connect's headline features, notification mirroring, replying to texts from your desktop, running commands on the phone, are **Android-only**. Apple's sandbox forbids a third-party app from reading other apps' notifications or sending SMS, so on iOS none of that works no matter what a generic comparison table says. What the **iOS** KDE Connect app (v0.5.x as of 2026) *does* deliver is real and useful: file transfer, **clipboard sync**, **battery status**, and media remote control. One more iOS quirk: Apple's background limits mean the KDE

Connect app must be **open or recently used** to stay reachable. It can't sit dormant for days and still answer.

9.1.1 LocalSend: your AirDrop replacement

Think of LocalSend as a dedicated screwdriver: it does exactly one thing, extremely well. Open it, pick a nearby device, send the file. No pairing, no account, no integration, and it cheerfully moves a 5 GB video that you'd never want to push through a more "integrated" tool.

That's its entire job, and it's the right tool whenever the task is purely "get this file from here to there."

9.1.2 KDE Connect: your phone companion

KDE Connect is the multitool. File transfer is just one of its features; the reason to install it is the *integration*, on iPhone specifically, that means a **shared clipboard** (copy `192.168.1.50` on your phone, paste it on your desktop) and **battery status** on your desktop panel. The notification/SMS superpowers are Android's; on iOS you're here for clipboard and file flow.

9.1.3 "If KDE Connect transfers files, why install LocalSend?"

You don't *have* to, plenty of people run only KDE Connect. The reason many keep LocalSend around anyway is sheer simplicity: when you just want to fling a big file across the room, LocalSend is two taps with zero ceremony. It's like keeping a dedicated screwdriver even though your multitool has one. Disk is cheap; install both and reach for whichever fits the moment.

9.2 Install them

9.2.1 On the iPhone

Both are free on the App Store: search **LocalSend** and **KDE Connect** (the KDE Connect one is published by KDE e.V.). Install both.

9.2.2 On your Linux laptop

LocalSend, easiest via Flatpak (works the same on Arch and Ubuntu):

```
flatpak install flathub org.localsend.localsend_app
```

(Or grab the AppImage from the project's GitHub releases if you don't use Flatpak.)

KDE Connect, from your distro's repos:

```
# Arch
sudo pacman -S kdeconnect

# Ubuntu/Debian
sudo apt install kdeconnect
```

Running KDE Connect on Hyprland (no Plasma, no GNOME)

KDE Connect does **not** require the full KDE Plasma desktop, but it does need its background daemon running and a way to interact with it. On a bare window manager like Hyprland:

- The daemon `kdeconnectd` is started on demand over D-Bus; launching the GUI (`kdeconnect-app`) or running `kdeconnect-cli -l` brings it up.
- For an at-a-glance tray/battery indicator, add a **Waybar** "custom" module that calls `kdeconnect-cli`, or just keep `kdeconnect-app` a keybind away.
- Skip **GSConnect**, it's a GNOME Shell extension and won't help you on Hyprland. Plain `kdeconnect` is the right choice here.

9.2.3 Open the firewall

Both tools need their ports reachable on the LAN. If you run `ufw`:

```
sudo ufw allow 53317/udp        # LocalSend discovery + transfer
sudo ufw allow 53317/tcp
sudo ufw allow 1714:1764/udp    # KDE Connect
sudo ufw allow 1714:1764/tcp
```

If a transfer "sees" the device but hangs at 0%, a closed firewall port is the first thing to check.

9.2.4 Pair

- **LocalSend:** no pairing, open it on both devices, on the same Wi-Fi, and they appear in each other's list. Send.
- **KDE Connect:** open the app on both, tap the laptop in the phone's device list, and **accept the pairing request** on the desktop (`kdeconnect-app` or the CLI will prompt). Pairing is a one-time trust handshake.

9.3 Making it work *anywhere*: over Tailscale

Here's the homelab tie-in. Out of the box, both tools find devices using **LAN multicast/broadcast discovery**, great at home, useless the moment your phone is on cellular, because broadcast packets don't traverse a VPN. But your tailnet gives every device a stable `100.x.y.z` address, and both apps let you **add a device by IP manually**, which sidesteps discovery entirely:

- **LocalSend:** Settings → **add a favorite / manual device** → enter the target's Tailscale IP (e.g. your laptop at `100.x.y.z`) with port `53317`. Now you can send to your laptop from the airport.
- **KDE Connect:** the iOS app has an **"Add device by IP"** / refresh option, enter the laptop's Tailscale IP. Pair once, and clipboard + file transfer work across the tailnet.

> Why this is the natural payoff of everything before it
>
> This is the same trick that powered the whole series: discovery and "same network" assumptions break the instant you leave home, and Tailscale's stable addresses are the fix. Just as you reached Audiobookshelf and `https://home.home` by tailnet IP, you reach your *laptop* by tailnet IP here. The phone Linux conveniences stop being a home-only luxury.

9.4 Phone the headless Pi

A caveat worth stating plainly: LocalSend and KDE Connect are **desktop** tools. Your Pi from Chapters 1–5 runs a **headless** server OS (Ubuntu Server 26.04 LTS), no graphical desktop, so neither app is the natural fit there. For moving a file between your phone and the Pi, you already have better, GUI-free paths:

- **Drop it into a service you already run.** Files you want *on* the Pi (more audiobooks, say) can go straight into `~/audiobookshelf/media/Audiobooks`, `scp` them over the tailnet:

  ```
  # from your laptop, over Tailscale, no LAN required
  scp ~/Downloads/lesson.mp3
  you@homelab:~/audiobookshelf/media/Audiobooks/
  ```

- **From the phone directly**, an iOS file-manager app that speaks SSH/SFTP (several exist) can connect to `homelab` over Tailscale and drop files onto the Pi without any desktop app in between.

In short: **laptop phone** is LocalSend/KDE Connect territory; **phone/laptop the headless Pi** stays SSH/`scp` territory, which you already have for free over Tailscale.

9.5 The wired fallback: USB access with libimobiledevice

Everything above is Wi-Fi. USB is a **completely separate technology stack**, and (honestly) most people rarely need it. Install it the day you actually hit one of these, not before:

- No Wi-Fi available and you need photos off the phone *now*.
- You're moving 100 GB of video and want the fastest possible transfer.
- The phone's networking is broken and you need emergency access.

When that day comes:

```
# Arch
sudo pacman -S libimobiledevice ifuse usbmuxd

# Ubuntu/Debian
sudo apt install libimobiledevice6 libimobiledevice-utils ifuse usbmuxd
```

Then plug in, trust the computer on the phone, and mount:

```
idevicepair pair                 # tap "Trust" on the iPhone when prompted
mkdir -p ~/iphone
ifuse ~/iphone                   # mounts the phone's media (camera roll)
# ... copy your files ...
fusermount -u ~/iphone           # unmount when done
```

> **i** What USB actually gives you (and what it doesn't)
>
> `usbmuxd` is the daemon that talks to the phone over the USB cable; `libimobiledevice` is the library implementing Apple's protocols; `ifuse` mounts the result as a folder. Because of iOS's sandbox, `ifuse` exposes the **photos/ camera-roll area** (and, with extra flags, individual apps' document folders), *not* the whole iOS filesystem. So USB is excellent for bulk photo/video offload, but it isn't a "browse my entire iPhone like a USB stick" experience. No jailbreak, no full filesystem, that's an Apple limitation, not a missing package.

9.6 If I were setting up your machine

Given your stack (iPhone, Hyprland, Arch/Ubuntu, plus the Pi homelab) start with exactly two things:

1. **LocalSend** (the AirDrop replacement)
2. **KDE Connect** (clipboard + battery + files)

…and nothing else. Add the Tailscale manual-IP pairing once you want these to work away from home. Then, *if* six months from now you find yourself saying "I wish I could pull my whole camera roll off over a cable," install `libimobiledevice` + `ifuse`, and at that point you'll understand exactly why you need them, instead of carrying tools you never touch.

For most people in 2026, Wi-Fi transfer (LocalSend / KDE Connect, extended over Tailscale) handles the overwhelming majority of phone Linux moments. USB is a backup and a power-user tool, not a daily requirement.

9.7 Recap: and the series, complete

- **LocalSend** = AirDrop replacement: dead-simple file send, install on both the iPhone and the laptop.

- **KDE Connect** = phone companion: clipboard sync and battery on iOS (the notification/SMS features are Android-only, don't expect them on iPhone).
- **Tailscale extends both off-LAN:** add devices by their `100.x.y.z` IP so transfer and clipboard work from anywhere, exactly as the rest of your homelab does.
- **The headless Pi** stays on SSH/`scp` over Tailscale, the right tool for a machine with no desktop.
- **USB (`libimobiledevice` + `ifuse`)** is the wired fallback for bulk photo offload or no-Wi-Fi emergencies, install it only when you actually need it.

That completes *Beginner Homelab on a Raspberry Pi*. You started by building a foundation (a hardened, always-on Pi on a private Tailscale network) and then gave it job after job: streaming your media, blocking ads on every device, clean `http://*.home` URLs behind a reverse proxy, a single `https://home.home` control panel, a deliberate answer for VPN privacy on the road, and now two-way file sharing with your phone. Every piece runs on hardware you own, over one private network, with nothing exposed to the public internet.

A Appendix A. The Docker Compose files, line by line

The main chapters keep the `compose.yaml` files terse on purpose. You don't need to understand every line to get a working homelab. This appendix is the opposite: it explains **every directive** in every Compose file the series creates, so when you come back in six months you can read your own stack and know exactly what each line does (and what's safe to change).

By the end of the series your Pi runs five containers across four Compose projects:

Project folder	File	Containers
`~/audiobookshelf`	`compose.yaml`	`audiobookshelf`
`~/pihole`	`compose.yaml`	`pihole`
`~/proxy`	`compose.yaml`	`caddy`
`~/dashboard`	`compose.yaml`	`homepage`, `portainer`

A.1 Compose concepts that show up everywhere

Before the per-file walkthroughs, here are the directives you'll see repeatedly. Read this once and the individual files become obvious.

- **`services:`**, the top-level map. Each key under it (e.g. `pihole:`) is one container Compose will manage.
- **`image:`**, which prebuilt image to pull and run, in `repository:tag` form. `pihole/pihole:latest` means "the `latest` tag of the `pihole/pihole` image." `:latest` floats; pinning a version (e.g. `:2024.07.0`) is more reproducible but you update by hand. This guide uses `:latest` for simplicity and updates with `docker compose pull`.
- **`container_name:`**, the fixed, human-friendly name (e.g. `pihole`). Without it, Compose auto-generates names like `pihole-pihole-1`. We set it explicitly because **the reverse proxy and the inter-container networking address each service by this exact name**, `reverse_proxy pihole:80` only works if the container is literally named `pihole`.
- **`ports:`**, publishes a **host** port to a **container** port, written `"HOST:CONTAINER"`. `"8081:80"` means "traffic arriving on the Pi's port 8081 is forwarded to port 80 inside the container." A bare `"80:80"` binds **all** the host's network interfaces (LAN *and* tailnet); prefixing an address, `"192.168.1.50:80:80"`, binds only that one interface. Ports are only needed for traffic entering from *outside* Docker, containers on the same Docker network reach each other directly without any `ports:` entry.
- **`environment:`**, variables set inside the container. This is how most images are configured. Values can be literals or `${VAR}` references resolved from the project's `.env` file (see below).

- `volumes:`, persistent storage. Two forms appear in this series:
  - **Bind mount**, `./etc-pihole:/etc/pihole` maps a *folder on the Pi* to a path inside the container. You can see and back up the files directly. The `./` is relative to the folder the `compose.yaml` lives in. A `:ro` suffix (`...:/etc/caddy/Caddyfile:ro`) mounts it **read-only**.
  - **Named volume**, `portainer_data:/data` stores data in a Docker-managed volume (declared in the top-level `volumes:` block). Use it when you don't need to touch the files yourself, only persist them across container recreations.
- `networks:`, which Docker networks the container joins. Containers on the same network resolve each other **by container name** as a hostname. This is the backbone of the reverse proxy: Caddy and the services share the `homelab` network, so `pihole`, `audiobookshelf`, and `homepage` are resolvable names.
- `restart:`, the restart policy. `unless-stopped` restarts the container on crash and on boot, *unless* you deliberately stopped it. `always` is similar but restarts even one you stopped, after a Docker daemon restart. Both are correct for an always-on Pi; the difference rarely matters.
- **Top-level `networks:` / `volumes:`**, declare resources the services reference. `external: true` means "this network already exists; don't create or destroy it, just attach to it", used for the shared `homelab` network that outlives any single project.

> **ℹ `${VAR}` interpolation, `.env`, and the `:?` / `:-` forms**
>
> Compose substitutes `${VAR}` from a file named `.env` in the same folder *before* starting the container. This is the mechanism that keeps secrets out of the file you commit:
>
> - `${PIHOLE_PASSWORD}`, plain substitution; empty if unset.
> - `${PIHOLE_PASSWORD:?set PIHOLE_PASSWORD in .env}`, **required**: Compose aborts with that error message if the variable is missing or empty. Used for secrets you must not launch without.
> - `${ABS_TOKEN:-}`, **default if unset**: the part after `:-` is the fallback (here, empty string), so a missing optional token doesn't error.
>
> The real values live in `.env`, which is listed in `.gitignore` and never leaves the Pi. The `compose.yaml` holds only references, so it's safe to publish.

A.2 `~/pihole/compose.yaml`: Pi-hole

```yaml
services:
  pihole:
    container_name: pihole
    image: pihole/pihole:latest

    ports:
      - "53:53/tcp"
      - "53:53/udp"
```

```yaml
      - "8081:80/tcp"

    environment:
      TZ: "${TZ}"
      FTLCONF_webserver_api_password: "${PIHOLE_PASSWORD:?set
PIHOLE_PASSWORD in .env}"
      FTLCONF_dns_upstreams: "1.1.1.1;1.0.0.1"
      FTLCONF_dns_listeningMode: "ALL"

    volumes:
      - ./etc-pihole:/etc/pihole

    networks:
      - homelab

    restart: unless-stopped

networks:
  homelab:
    external: true
```

- **`ports: 53:53/tcp` and `53:53/udp`**, DNS uses **both** transports (UDP for almost everything, TCP for large responses), so both are published. They're bound to all interfaces so the Pi answers DNS for your LAN *and* over the tailnet. (In Chapter 3, before the proxy existed, these were bound to the Pi's LAN IP only, `${PIHOLE_IP}:53:53`; Chapter 4 widens them.)
- **`ports: 8081:80/tcp`**, Pi-hole's web UI listens on port 80 *inside* the container; we publish it on the host as **8081** so the host's port 80 stays free for Caddy.
- **`TZ`**, the timezone, so Pi-hole's logs and graphs use your local time.
- **`FTLCONF_*`**, Pi-hole v6's configuration system. Each variable maps to a key in `/etc/pihole/pihole.toml` by the rule `FTLCONF_<section>_<key>` → `[section] key`. Anything set this way is **re-applied on every container start** and shows as read-only in the web UI, exactly what you want for file-defined, reproducible config:
  - `FTLCONF_webserver_api_password`, the admin/API password (the v6 successor to the old `WEBPASSWORD`).
  - `FTLCONF_dns_upstreams`, the real resolvers Pi-hole forwards *non-blocked* queries to; `1.1.1.1;1.0.0.1` is Cloudflare's primary + secondary.
  - `FTLCONF_dns_listeningMode: "ALL"`, answer queries on any interface. Required for a Dockerized Pi-hole, because LAN clients' queries arrive *through* the Docker gateway and the stricter "local-only" mode would reject them.
- **`volumes: ./etc-pihole:/etc/pihole`**, persists Pi-hole's entire state (config, blocklists, query database, your local DNS records) in a folder next to the compose file, so recreating the container loses nothing.
- **`networks: homelab` + top-level `external: true`**, joins the shared proxy network *declaratively*. This is deliberate: attaching it by hand with `docker network connect` would be wiped by the next `--force-recreate` (see the warning in Chapter 4). In the file, it always re-attaches.

A.3 `~/proxy/compose.yaml` + `Caddyfile`: the reverse proxy

```yaml
services:
  caddy:
    container_name: caddy
    image: caddy:latest
    ports:
      - "80:80"
      - "443:443"
      - "443:443/udp"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    networks:
      - homelab
    restart: unless-stopped

networks:
  homelab:
    external: true

volumes:
  caddy_data:
  caddy_config:
```

- **`ports: 80:80, 443:443, 443:443/udp`**, Caddy is the **single front door**, so no other container may publish these. Port **80** catches plain-HTTP requests and redirects them to HTTPS; port **443** serves HTTPS (the `/udp` line enables HTTP/3). Every `*.home` request lands here first.
- **`volumes`**:
  - `./Caddyfile:/etc/caddy/Caddyfile:ro`, mounts your routing table into the container, **read-only** (Caddy never needs to write its own config).
  - `caddy_data` / `caddy_config`, named volumes for Caddy's internal state. Crucially, `caddy_data` holds the **internal certificate authority** that `tls internal` creates (`/data/caddy/pki/...`) and the certs it signs, so your trusted CA survives container recreations. Kept as named volumes because you never edit them by hand.
- **`networks: homelab`**, lets Caddy reach each backend by container name. This is the whole trick: Caddy connects to `pihole:80`, `audiobookshelf:80`, `homepage:3000` *over this network*, completely ignoring the host port mappings. That's why moving Pi-hole's host UI port to 8081 doesn't affect Caddy.

The companion `Caddyfile` is the routing table:

```
pihole.home {
    tls internal
    redir / /admin 302
    reverse_proxy pihole:80
}

abs.home {
    tls internal
    reverse_proxy audiobookshelf:80
}

home.home, homelab {
    tls internal
    reverse_proxy homepage:3000
}
```

- **`{ ... }` site block**, one per hostname (or comma-separated list of hostnames). The address is a **bare name** (no `http://`/`https://`); Caddy matches the incoming request's `Host` header to the right block.
- **`tls internal`**, the key directive. There's no public certificate authority for a private `.home` name, so Caddy spins up its **own** local CA, signs a cert for each name, and serves HTTPS. You trust that CA once per device (Chapter 4 Step 7) and the URLs get a real padlock. Caddy also auto-redirects `http://` to `https://` for these names.
- **`reverse_proxy <name>:<port>`**, forwards the request to that container over the `homelab` network. The port is the service's *internal* port, not its published host port.
- **`redir / /admin 302`**, Pi-hole's UI lives under `/admin`; this sends a bare `https://pihole.home/` to `https://pihole.home/admin` so you land in the right place. The `302` is a temporary-redirect status code.
- **`home.home, homelab`**, two names, one backend: both open the dashboard.

A.4 `~/dashboard/compose.yaml`: Homepage + Portainer

```yaml
services:
  homepage:
    image: ghcr.io/gethomepage/homepage:latest
    container_name: homepage
    volumes:
      - ./config:/app/config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    environment:
      HOMEPAGE_ALLOWED_HOSTS: home.home,homelab,homelab.your-tailnet.ts.net
      HOMEPAGE_VAR_PIHOLE_PASSWORD: ${PIHOLE_PASSWORD:?set PIHOLE_PASSWORD
in .env}
      HOMEPAGE_VAR_ABS_TOKEN: ${ABS_TOKEN:-}
```

```yaml
    networks:
      - homelab
    restart: unless-stopped

  portainer:
    image: portainer/portainer-ce:lts
    container_name: portainer
    ports:
      - "9443:9443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data
    networks:
      - homelab
    restart: always

networks:
  homelab:
    external: true

volumes:
  portainer_data:
```

This is the only project with **two** services. Note Homepage has **no `ports:`** , it's reached only through Caddy (`homepage:3000` over `homelab`), so it never touches a host port.

Homepage:

- **`image: ghcr.io/gethomepage/homepage:latest`**, the full registry path; this one lives on GitHub's Container Registry (`ghcr.io`) rather than Docker Hub.
- **`volumes`**:
  - `./config:/app/config`, your dashboard's YAML config (tiles, widgets, bookmarks) on the Pi, editable directly.
  - `/var/run/docker.sock:/var/run/docker.sock:ro`, the **Docker socket**, **read-only**. This lets Homepage read container status (the live "running" dot, CPU/RAM) without being able to control Docker.
- **`HOMEPAGE_ALLOWED_HOSTS`**, a required allow-list of hostnames Homepage will render for. It must contain *every* name Caddy forwards here (`home.home`, `homelab`, and the Pi's full tailnet name). A request for a name not on the list gets a blank page, the single most common Homepage gotcha. **This variable only takes effect on a container recreate** (`up -d --force-recreate`), not a plain `restart`.
- **`HOMEPAGE_VAR_*`**, Homepage exposes any `HOMEPAGE_VAR_<NAME>` variable to its config files as `{{HOMEPAGE_VAR_<NAME>}}`. This is how widget secrets (the Pi-hole password, the Audiobookshelf API token) reach the config **without being written into the config files**. They come from `.env` at runtime.

Portainer:

- **`image: portainer/portainer-ce:lts`**, Community Edition, Long-Term Support tag. Runs natively on the 64-bit Pi.
- **`ports: 9443:9443`**, Portainer's own HTTPS UI (self-signed certificate). It keeps a host port because you reach it directly at `https://homelab:9443`, not through Caddy.
- **`volumes`**:
  - `/var/run/docker.sock:/var/run/docker.sock`, the Docker socket, **read-write this time** (no `:ro`). Portainer's whole job is to *control* Docker, start, stop, recreate, pull, so it needs full socket access.
  - `portainer_data:/data`, a named volume for Portainer's own database (users, settings).
- **`restart: always`**, Portainer is your break-glass management console; you want it back even after a manual stop + daemon restart.

> ⚠ The Docker socket is root-equivalent
>
> Mounting `/var/run/docker.sock` into a container gives that container control over Docker, which on Linux is effectively root on the host. That's why Homepage gets it **`:ro`** (it only needs to *read* status) and only Portainer gets it read-write (it needs to *manage*). Never expose a container that mounts the socket to the public internet; on your tailnet-only homelab it's fine.

A.5 `~/audiobookshelf/compose.yaml`: Audiobookshelf

The media server set up in Chapter 2. It's the first stack you create, then Chapter 4 adds the `networks` block so Caddy can reach it as `audiobookshelf:80`:

```yaml
services:
  audiobookshelf:
    container_name: audiobookshelf
    image: ghcr.io/advplyr/audiobookshelf:latest

    ports:
      - "13378:80"

    volumes:
      - ./media/Audiobooks:/audiobooks
      - ./config:/config
      - ./metadata:/metadata

    networks:
      - homelab

    restart: unless-stopped

networks:
  homelab:
    external: true
```

The whole stack is self-contained in `~/audiobookshelf/`: the `compose.yaml`, the app's `config/` and `metadata/`, and the content under `media/`. All the volume paths are relative to that folder.

- **`ports: 13378:80`**, Audiobookshelf listens on port 80 *inside* the container; we publish it on the host as **13378** (host 80 is reserved for Caddy). You still reach the app directly at `homelab:13378`, the iPhone app uses that address, while the browser uses `abs.home` through Caddy.
- **`volumes`**:
  - `./media/Audiobooks:/audiobooks`, your audiobook library. A future Podcasts library is just another folder, `./media/Podcasts:/podcasts`. (Audiobookshelf is for spoken audio; music gets its own server.)
  - `./config:/config` and `./metadata:/metadata`, Audiobookshelf's database (users, listening positions, settings) and cached cover art, kept next to the compose file so recreating the container loses nothing.
- **`networks: homelab` + top-level `external: true`**, joins the shared proxy network *declaratively*, exactly like Pi-hole. Added in Chapter 4; before that the stack runs on its own default network. Declaring it here (rather than a hand `docker network connect`) is what makes the attachment survive every recreate, see the warning under Pi-hole above.

B Appendix B. What should just work, and how to verify it

This is the at-a-glance reference for a finished build: every URL, what it's for, and the one-line check that proves it's healthy. Use it as a smoke test after a reboot, after an update, or any time something feels off, work top to bottom and the first failing row tells you which layer broke.

Throughout, substitute your own values where this guide uses placeholders: the Pi's Tailscale IP for `100.x.y.z` (run `tailscale ip -4` on the Pi) and your tailnet suffix for `your-tailnet.ts.net`.

B.1 The one-time cloud setting everything depends on

Almost everything below is configured *on the Pi* and works the moment the containers are up. The **single exception** is a setting in the Tailscale admin console that can't be done from the Pi, and until it's on, the `.home` names and tailnet-wide ad-blocking won't work on your *other* devices:

! Make Pi-hole your tailnet's DNS (admin console, one time)

In the Tailscale admin console (https://login.tailscale.com) → **DNS** page:

1. **Add nameserver → Custom**, enter the Pi's **Tailscale** IP (`100.x.y.z`), leave *Restrict to domain* **off**, and *Use with exit node* **off**. Save.
2. Turn **on "Override local DNS."**

This is the Chapter 4 Step 5 step. Verify it took with `tailscale dns status` on any device, under *Resolvers* you should see your Pi's `100.x.y.z`, not "no resolvers configured." If it still says no resolvers, the admin-console setting hasn't applied yet.

B.2 The service map

Service	URL (everywhere on your tailnet)	What it does
Dashboard (Homepage)	`https://home.home` (or `https://homelab`)	Landing page; live tiles + links
Audiobookshelf	`https://abs.home`	Stream your audiobooks / Assimil
Pi-hole	`https://pihole.home`	DNS ad-blocking admin
Portainer	`https://homelab:9443`	Docker management GUI

Service	URL (everywhere on your tailnet)	What it does

All four work from any device signed into your tailnet, home Wi-Fi or cellular, with no IPs and no port numbers (except Portainer, which keeps its own `:9443`).

B.3 Verify it, layer by layer

Run these in order. Each isolates one layer, so the first failure pinpoints the problem.

1. The containers are all up (on the Pi):

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

You should see five **Up** containers: `caddy`, `homepage`, `pihole`, `portainer`, `audiobookshelf`.

2. They share the proxy network (on the Pi):

```
docker network inspect homelab --format "{{range .Containers}}{{.Name}}
{{end}}"
```

All five names should appear. If `pihole` or `audiobookshelf` is missing, the reverse proxy can't reach it, re-read the network callout in Chapter 4.

3. DNS resolves the pretty names (on the Pi):

```
dig +short pihole.home @127.0.0.1     # expect your Pi's Tailscale IP
(100.x.y.z)
dig +short doubleclick.net @127.0.0.1 # expect 0.0.0.0 (ad-blocking works)
```

4. The proxy routes each name to the right service (on the Pi):

```
for name in pihole.home abs.home home.home; do
  printf "%-12s -> " "$name"
  curl -s -o /dev/null -w "HTTP %{http_code}\n" \
    --resolve "$name:80:$(tailscale ip -4 | head -1)" "http://$name/"
done
```

Expect `pihole.home -> HTTP 302` (it redirects to `/admin`) and `abs.home` / `home.home -> HTTP 200`. This tests the full chain, DNS name → Caddy on the Tailscale IP → the correct backend container, exactly as a remote device sees it.

5. The names work from another device (laptop/phone on the tailnet):

```
ping home.home          # answers from the Pi's 100.x.y.z
```

Then open `https://home.home`, `https://abs.home`, and `https://pihole.home` in a browser. If `ping` fails here but step 4 passed on the Pi, the missing piece is the **Tailscale DNS override** (the cloud setting at the top of this appendix).

6. The dashboard widgets have live data:

Open `https://home.home`. The Pi-hole tile should show today's blocked-query count and the Audiobookshelf tile your library, proof the widgets authenticated through the `homelab` network using the secrets in `~/dashboard/.env`. If a tile says "API error," see the widget troubleshooting in Chapter 5 (for Pi-hole, the usual culprit is a missing `version: 6`).

B.4 What works away from home (and what deliberately doesn't)

Once the cloud setting above is on, here's the honest picture the moment you leave the house, the full reasoning is in Chapter 8:

Mode (away)	Homelab URLs?	Pi-hole ad-block?	Hides your IP?
Tailscale on, no exit node (everyday default)			
Tailscale + your Pi as exit node			(home IP)
Tailscale + Mullvad exit node		(Mullvad DNS)	
Aura (or any standalone VPN) on			

The one rule behind the table: a phone runs **one** VPN at a time, and whatever VPN is active owns DNS. So homelab access, Pi-hole filtering, and a privacy VPN can't all be on at once. You switch between coherent modes. The everyday default (top row) is the one you live in.

B.5 Reboot behavior

Nothing extra is required after a power cycle. Every container uses a `restart:` policy (`unless-stopped` or `always`), so Docker brings them all back on boot; the `homelab` network is persistent; and the `systemd-resolved` stub stays disabled (the drop-in from Chapter 3 survives reboots), so Pi-hole reclaims port 53 cleanly. Reboot the Pi and re-run the verification steps above, every row should pass without you touching anything.